

AUTOFIX

INTELLIGENT MULTI BRANCH VEHICLE SERVICE HUB

ABDUL WASAY

AHMAD ALI



DEPARTMENT OF COMPUTER SCIENCES
COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
WAH CANTT – PAKISTAN

SESSION 2022-2026

AUTOFIX

INTELLIGENT MULTI BRANCH VEHICLE SERVICE HUB

Undertaken By:

Abdul Wasay

REG. NO. CIIT/FA22-BCS-127/WAH

Ahmad Ali

REG. NO. CIIT/FA22-BCS-144/WAH

Supervised By:

Dr. Mamuna Fatima



A DISSERTATION SUBMITTED AS A PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELORS IN COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCES
COMSATS UNIVESITY ISLAMABAD, WAH CAMPUS
WAH CANTT – PAKISTAN

SESSION 2022-2026

FINAL APPROVAL

The final approval page will be provided by the department after the final evaluation.

DEDICATION

We dedicate this project firstly to Allah Almighty, whose grace and guidance have sustained us throughout this journey. Our deepest gratitude extends to our loving parents, whose unconditional support and sacrifices have paved the way for our achievements. We are eternally grateful to our esteemed supervisor, Dr. Mamuna Fatima, for her mentorship, wisdom, and unwavering belief in our potential. We sincerely thank our dedicated teachers, who have nurtured our growth and inspired us to strive for excellence. Finally, we acknowledge our cherished friends and siblings, whose unwavering companionship, encouragement, and unwavering support have been a constant source of strength.

ACKNOWLEDGEMENT

Our deepest gratitude goes to Almighty Allah, whose boundless mercy and guidance have sustained us throughout this project and our academic journey. We honor the Prophet Muhammad (Peace Be Upon Him), whose teachings illuminate our path.

We are eternally grateful to our parents, whose love and sacrifices have made our achievements possible. We extend heartfelt thanks to our teachers, especially Dr. Mamuna Fatima, for her unwavering support, mentorship, and encouragement. Her guidance has been invaluable, and we carry her lessons with us.

PROJECT BRIEF

PROJECT NAME	AUTOFIX Intelligent Multi-Branch vehicle Service Hub
ORGANIZATION NAME	COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
OBJECTIVE	TO MODERNIZE VEHICLE SERVICE MANAGEMENT THROUGH AN AI-POWERED PLATFORM ENABLING SMART BOOKINGS, PREDICTIVE MAINTENANCE, AND EFFICIENT MULTI-BRANCH OPERATIONS FOR CUSTOMERS AND MANAGERS.
UNDERTAKEN BY	ABDUL WASAY AHMAD ALI
SUPERVISED BY	DR. MAMUNA FATIMA DOCTOR COMPUTER SCIENCE DEPARTMENT COMSATS UNIVERSITY ISLAMABAD, WAH CAMPUS
STARTED ON	NOVEMBER 2025
COMPLETED ON	APRIL 2026
COMPUTER USED	INTEL CORE I7 11TH GENERATION
SOURCE LANGUAGE	PYTHON, REACT, JAVA SCRIPT, NODE JS
OPERATING SYSTEM	MICROSOFT WINDOWS 10
TOOLS USED	VISUAL STUDIO CODE, JUPYTER

ABSTRACT

AutoFix is an intelligent web-based platform designed to modernize vehicle service management for multi-branch automotive service centers. The system connects customers, managers, and administrators through a centralized platform that streamlines service booking, inventory management, and service tracking. Customers can schedule appointments, receive AI-driven maintenance recommendations based on vehicle data, and track service progress in real time. Managers can monitor mechanic workloads, manage spare parts inventory across branches, and access analytical dashboards for operational insights. By integrating predictive maintenance, automated notifications, and centralized analytics, AutoFix improves customer experience, enhances operational efficiency, and enables data-driven decision making for service centers.

TABLE OF CONTENTS

1	INTRODUCTION	1
1.1	System Introduction	1
1.2	Background of the System	2
1.3	Objectives of the System	3
1.4	Significance of the System.....	4
2	REQUIREMENT SPECIFICATIONS.....	6
2.1	Product Scope	6
2.2	Product Description	6
2.2.1	Product Perspective.....	6
2.2.2	Product Functionality	8
2.2.3	Users and Characteristics	8
2.2.4	Operating Environment.....	9
2.3	Specific Requirements	10
2.3.1	Functional Requirements	10
2.3.2	Behavioral Requirements.....	14
2.3.3	External Interface Requirements.....	15
2.4	Non-functional Requirements.....	18
2.4.1	Performance Requirements.....	18
2.4.2	Safety and Security Requirements	18
2.4.3	Software Quality Attributes	19
3	DESIGN SPECIFICATIONS.....	21
3.1	Introduction.....	21
3.2	Composite Viewpoint	21
3.3	Logical Viewpoint	22
3.4	Information Viewpoint.....	23
3.5	Interaction Viewpoint	24
3.6	State Dynamics Viewpoint	27
3.7	Algorithmic Viewpoint	28
3.7.1	AI Maintenance Prediction Algorithm.....	28

3.7.2	Booking Creation Algorithm	28
4	DEVELOPMENT AND TOOLS	30
4.1	Introduction.....	30
4.2	Development Plan.....	30
4.2.1	Development Phases:	31
4.2.2	Development Timeline (Gantt Chart):	31
4.3	Development Tools.....	33
4.4	Conclusion and Future Work/Extensions	34
5	QUALITY ASSURANCE.....	35
5.1	Introduction.....	35
5.2	Traceability Matrix	35
5.3	Test Plan.....	36
6	USER MANUAL	40
6.1	Introduction.....	40
6.2	Hardware/Software Requirements for the System.....	40
6.3	Installation guide for Application	40
6.4	Operating Manual	42
7	REFERENCES	54

LIST OF FIGURES

Figure 2.1: General System Diagram	7
Figure 2.2: Use Case Diagram	14
Figure 2.3: Admin Dashboard Overview	15
Figure 2.5: Customer Dashboard overview	16
Figure 3.1: Package Diagram.....	21
Figure 3.2: Class Diagram.....	22
Figure 3.3: ERD Diagram	23
Figure 3.4: Admin Sequence Diagram.....	24
Figure 3.5: Manager Sequence Diagram.....	25
Figure 3.6: Customer Sequence Diagram	26
Figure 3.7: Booking Process State Diagram	27
Figure 6.1: Home Page (Hero Section Screen).....	42
Figure 6.2: Vehicle Services Page Screen.....	43
Figure 6.3: Branches / Service Centers Screen	44
Figure 6.4: Book Service Appointment Screen	45
Figure 6.5: Contact Us Page Screen	46
Figure 6.6: Login Screen	47
Figure 6.7: Vehicle Maintenance Prediction Tool Screen.....	48
Figure 6.8: AI Assistant Chatbot Screen.....	49
Figure 6.9: Payment Modal Screen.....	50
Figure 6.10: Admin Dashboard Sidebar	51
Figure 6.11: Manager Dashboard Sidebar	52
Figure 6.12: Customer Dashboard Sidebar	53

LIST OF TABLES

Table 2.1: Users and Characteristics.....	8
Table 2.2: Functional Requirements (User Authentication and Profile Management)	10
Table 2.3: Functional Requirements (Service Booking and Appointment Management).....	10
Table 2.4: Functional Requirements (AI Customer Assistant and Recommendation Engine).....	11
Table 2.5: Functional Requirements (Multi-Branch Service and Inventory Management)	11
Table 2.6: Functional Requirements (Vehicle Service Tracking and Task Allocation	12
Table 2.7: Functional Requirements (Centralized Analytics and Dashboard System).....	12
Table 2.8: Functional Requirements (Notification and Activity Monitoring System)	13
Table 2.9: Functional Requirements (Customer Feedback and Blog Management)	13
Table 4.1: Development Phases	31
Table 5.1: Traceability Matric for AutoFix	35
Table 5.2: Test Case TC-1 (User Registration)	36
Table 5.3: Test Case TC-2 (User Login)	36
Table 5.4: Test Case TC-3 (Service Booking).....	37
Table 5.5: Test Case TC-4 (AI Maintenance Prediction)	37
Table 5.6: Test Case TC-5 (Inventory Management)	38
Table 5.7: Test Case TC-6 (Service Tracking)	38
Table 5.8: Test Case TC-7 (Analytics Dashboard).....	39
Table 5.9: Test Case TC-8 (Feedback System)	39

1 INTRODUCTION

This chapter introduces the AutoFix system and provides an overview of the problem domain, system objectives, and the significance of the proposed solution. It explains the motivation behind developing the system and how it addresses the operational challenges faced by multibranch vehicle service centers. The chapter also highlights the system's purpose, its background within the automotive service management domain, and the goals the system aims to achieve. By presenting the objectives and importance of AutoFix, this chapter establishes the foundation for understanding the detailed system requirements described in the following chapters.

1.1 System Introduction

AutoFix is a web-based intelligent vehicle service management platform specifically designed to support the complex operational requirements of multi-branch automotive service centers. The system serves three primary categories of users: customers seeking to schedule and track vehicle maintenance services, branch managers responsible for overseeing daily operations and staff workloads, and system administrators who manage the broader configuration and coordination of the platform across multiple branches. For customers, AutoFix offers a convenient online interface that allows them to register, log in, book service appointments by selecting their preferred branch and available time slots, submit vehicle information, and monitor the real-time status of their service requests. Customers also receive automated notifications via email or SMS at key stages of the service lifecycle, such as booking confirmation, service commencement, and completion. For managers and administrators, the platform provides a comprehensive set of tools to oversee branch performance, track mechanic task allocations, manage spare parts inventory across warehouses, and generate detailed analytics reports on bookings, revenue, and operational efficiency. A distinguishing feature of AutoFix is its integration of artificial intelligence for predictive vehicle maintenance. Using a Random Forest classifier trained on historical vehicle data, the system analyzes vehicle parameters such as mileage, engine hours, fuel efficiency, and component condition indicators to predict the likelihood of requiring specific maintenance services. This

AI-driven recommendation engine helps customers make informed decisions about their vehicle care and assists service centers in preparing appropriate resources and staff in advance. The system is built on a client-server architecture, with a React-based single-page application on the front end communicating with a Django REST API on the back end. MySQL serves as the primary relational database, Redis is used for session caching and performance optimization, and the machine learning models are deployed as integrated Python services within the backend. Overall, AutoFix is designed to digitize, centralize, and intelligently enhance every aspect of automotive service management, from the first customer interaction to the final service report.

1.2 Background of the System

The global automotive industry is one of the largest and most economically significant sectors in the world, with hundreds of millions of vehicles requiring regular maintenance and repair services each year. Despite this scale, the operational infrastructure of vehicle service centers has largely remained manual and fragmented. Traditional service centers typically rely on physical appointment books, telephone-based scheduling, handwritten job cards, and informal communication between staff members. Spare parts inventory is often managed through standalone spreadsheets or even paper logs, making it difficult to maintain accurate stock levels across multiple locations. Task assignment to mechanics is frequently handled verbally or through informal whiteboards, with no systematic tracking of workload distribution or task completion. These conventional methods introduce a wide range of operational problems. Double bookings arise when appointment records are not synchronized across staff members. Inventory shortages occur due to the absence of automated restocking alerts, leading to delays in service delivery. Customers receive little to no transparency regarding the status of their vehicles once they are handed over for service, and pricing information is often communicated inconsistently. For service centers operating across multiple branches, coordination becomes especially challenging, as there is no centralized system for monitoring performance, sharing resources, or maintaining consistent service standards. From the customer's perspective, identifying the appropriate maintenance for their vehicle is also a significant challenge. Most vehicle owners are not technically trained and rely solely on the judgment of mechanics, which can lead to over-servicing or under-servicing of their vehicles. The absence of data-

driven maintenance recommendations means that vehicle health is often assessed reactively rather than proactively. While some commercial automotive management software solutions exist, they tend to address only isolated aspects of service center operations, such as basic appointment scheduling or simple billing. Few systems provide comprehensive coverage of all operational domains, and fewer still incorporate intelligent features such as machine learning-based maintenance prediction, centralized multi-branch analytics, or real-time service tracking. AutoFix is developed in response to these identified gaps. By combining appointment management, AI-powered maintenance recommendations, multi-branch inventory control, real-time service tracking, and analytics reporting into a single integrated platform, AutoFix provides a holistic and intelligent solution tailored to the needs of modern automotive service operations. The system is designed not only to improve internal efficiency but also to enhance the transparency and convenience of the service experience for customers.

1.3 Objectives of the System

- To develop a centralized web-based platform that enables unified management of vehicle service operations across multiple branches, eliminating the need for isolated or branch-specific tools and ensuring consistency in workflows, data, and communication throughout the organization.
- To provide customers with a convenient, accessible online appointment booking system that displays real-time slot availability for their chosen branch, allowing them to select preferred service dates and times, submit vehicle details, and receive instant booking confirmations without requiring telephone or in-person contact.
- To implement an artificial intelligence-based vehicle maintenance recommendation system that uses a Random Forest machine learning classifier to analyze vehicle parameters such as mileage, engine hours, fuel efficiency, and component condition data, and predict the specific maintenance services a vehicle is likely to require, enabling proactive rather than reactive servicing.
- To provide branch managers with real-time visibility into staff workload distribution, ongoing service progress, mechanic task assignments, and overall branch performance metrics, enabling more effective resource allocation, faster identification of bottlenecks, and improved operational decision-making.

- To implement a centralized spare parts inventory management system that tracks component stock levels across all branches and warehouses, triggers automated low-stock alerts when inventory falls below defined thresholds, and facilitates timely restocking to prevent service delays caused by parts unavailability.
- To provide administrators and managers with centralized, interactive analytics dashboards that aggregate and visualize key performance indicators including total bookings, revenue figures, service completion rates, customer satisfaction scores, and inventory turnover, enabling data-driven strategic planning and performance benchmarking across branches.
- To enhance transparency and customer engagement by implementing an automated notification system that sends timely updates to customers via email or SMS at each critical stage of the service lifecycle, including booking confirmation, service initiation, parts replacement notifications, and service completion alerts, ensuring customers are always informed of their vehicle's status.

1.4 Significance of the System

The AutoFix system holds considerable significance both in terms of its practical utility for the automotive service industry and its broader contribution to the digitalization of service management practices. At the operational level, AutoFix eliminates many of the inefficiencies that characterize traditional service center management by replacing manual, error-prone processes with a centralized, automated digital platform. By digitizing appointment scheduling, mechanics can be assigned tasks more efficiently, and service managers can gain a real-time overview of workload distribution and branch performance. Automated inventory alerts ensure that spare parts are reordered before stock levels become critical, reducing service delays caused by unavailability of components. From a customer perspective, AutoFix significantly improves the service experience by providing transparency, convenience, and proactive support. Customers can book appointments at any time without needing to call the service center, receive automated updates at every stage of the service process, and benefit from AI-driven maintenance recommendations that help them understand and anticipate their vehicle's needs. This level of engagement and transparency is increasingly expected by modern consumers and represents a meaningful improvement over the communication

standards of traditional service centers. From a business intelligence standpoint, the analytics dashboards provided by AutoFix enable service center owners and administrators to make data-driven decisions regarding resource allocation, staffing, pricing strategies, and branch expansion. The ability to monitor key performance indicators such as booking volumes, revenue trends, customer satisfaction ratings, and inventory turnover across all branches from a single interface provides a significant strategic advantage. The applicability of AutoFix extends across a variety of organizational contexts within the automotive sector. The system is well-suited for independent multi-branch vehicle service centers, automobile dealership service departments, corporate fleet maintenance operations, and automotive workshop chains. In each of these contexts, the platform's modular design and role-based access control ensure that it can be adapted to the specific workflows and user requirements of the organization. In summary, AutoFix represents a meaningful step forward in the modernization of automotive service management. By integrating intelligent technology with practical operational tools, the system not only improves the efficiency and reliability of service center operations but also contributes to a more satisfying and transparent experience for vehicle owners. Its development is motivated by a clear need in the industry and grounded in a thorough understanding of the challenges faced by both service providers and their customers.

2 REQUIREMENT SPECIFICATIONS

This chapter describes the functional scope and overall requirements of the AutoFix system. It explains what the system is expected to do and the boundaries within which it will operate. The chapter also provides an overview of the system's environment, its users, and the major functionalities offered by the platform. By defining the product scope, system perspective, user characteristics, and operating environment, this chapter establishes a clear understanding of how the AutoFix system will function and interact with its users and supporting technologies

2.1 Product Scope

AutoFix is a web-based intelligent vehicle service management system designed for multi-branch automotive service centers. The platform provides a centralized environment where customers can book vehicle service appointments, receive AI-driven maintenance recommendations, and track service progress in real time. The system also enables service center managers to manage branch operations, allocate tasks to mechanics, track spare parts inventory, and monitor operational performance through analytics dashboards.

The scope of AutoFix includes online appointment booking, predictive maintenance recommendations using machine learning, centralized inventory management, service tracking, notifications, and customer feedback management. The system focuses specifically on digitalizing service center operations and improving coordination between customers and service providers. However, the system does not perform physical vehicle diagnostics, repair operations, or direct financial transaction processing. Its primary role is to facilitate service management, decision support, and communication within automotive service centers.

2.2 Product Description

2.2.1 Product Perspective

AutoFix is a standalone web-based application developed to address the operational challenges faced by traditional vehicle service centers. Many existing service centers rely on manual recordkeeping, fragmented systems, or basic scheduling tools that do not support multi-branch coordination or intelligent decision-making. AutoFix replaces these traditional processes with a centralized digital platform that integrates booking

management, inventory tracking, staff allocation, and predictive maintenance recommendations.

The system operates within a client-server architecture where users access the application through web browsers while the backend services manage business logic, data storage, and analytics. The platform integrates several technologies including a React-based user interface, Django REST APIs for backend services, a MySQL database for structured data storage, Redis for caching, and machine learning models implemented using Python. The system interacts with external services such as email and SMS gateways to send notifications and alerts. Through these integrations, AutoFix provides a seamless environment for managing automotive service operations across multiple branches.

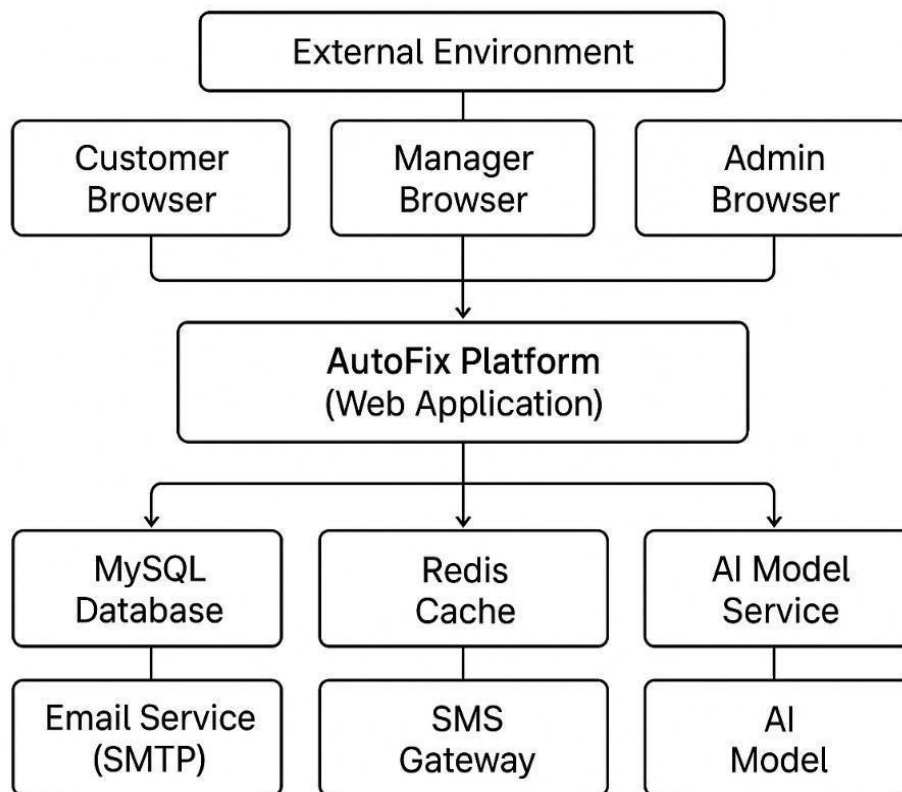


Figure 2.1: General System Diagram

2.2.2 Product Functionality

Below are the core product functionalities that the AutoFixct system provides:

- User authentication and profile management for customers, managers, and administrators
- Online vehicle service booking with real-time slot availability
- AI-based vehicle maintenance prediction and service recommendations
- Multi-branch service management and inventory tracking for spare parts
- Vehicle service tracking and mechanic task allocation
- Centralized analytics dashboards displaying operational insights
- Notification and activity monitoring for bookings, inventory alerts, and service updates
- Customer feedback and rating system to evaluate service quality
- Blog and informational content management for vehicle care tips

2.2.3 Users and Characteristics

Table 2.1: Users and Characteristics

User Role	Primary Domain	Core Responsibilities
Customer	Service Seeker / Vehicle Owner	Register and manage personal profiles (with multiple vehicles), book service appointments, view service pricing, receive AI-based maintenance recommendations, track service status, and provide feedback/ratings.
Manager	Branch Operations / Local Oversight	Manage branch profiles and services, track mechanic workload and allocate tasks, monitor multi-branch inventory, view analytics and reports, and handle customer complaints.

Administrator	System-Wide Control / Configuration	Manage user accounts and role assignments, configure system settings and business rules, oversee all branches and performance metrics, and manage content (blogs, notifications).
---------------	-------------------------------------	---

2.2.4 Operating Environment

- **Hardware Platform**
 - Runs on standard devices without specialized hardware
 - Dual-core processor with minimum 2.0 GHz speed
 - Minimum 4 GB RAM required for smooth operation
- **Operating System Support**
 - Supports Windows 10 and later operating systems
 - Android devices supported via mobile web browsers
- **Web Browser Compatibility**
 - Supports Google Chrome version 90 or higher
 - Supports Microsoft Edge version 90 or newer
- **Frontend Technologies**
 - React.js framework used for interactive user interfaces
 - HTML5, CSS3, and JavaScript for frontend development
 - Chart.js library used for analytics data visualization
- **Backend Technologies**
 - Django REST Framework used for API development
 - Python handles server-side logic and AI integration

- **Database and Caching**
 - MySQL database used for structured data storage
 - Redis used for caching and session management
 - AI and Data Processing
 - Scikit-learn library used for Random Forest model

2.3 Specific Requirements

2.3.1 Functional Requirements

Table 2.2: Functional Requirements (User Authentication and Profile Management)

Req. ID	Description
FR01	The system shall allow users to register by providing personal details including name, email address, phone number, and password.
FR02	The system shall authenticate users through a secure login process using registered credentials.
FR03	The system shall generate and validate secure authentication tokens using JWT (JSON Web Tokens).
FR04	The system shall allow users to recover forgotten passwords through email verification.
FR05	The system shall allow customers to manage their personal profile information including contact details and profile pictures.
FR06	The system shall allow customers to register multiple vehicles by providing vehicle details such as make, model, year, mileage, and VIN number.

Table 2.3: Functional Requirements (Service Booking and Appointment Management)

Req. ID	Description
FR07	The system shall display available services and their pricing for each branch.
FR08	The system shall allow customers to select a service type, preferred branch, and available time slot.

FR09	The system shall validate time slot availability and branch working hours before confirming a booking.
FR10	The system shall generate a booking confirmation after successful appointment scheduling.
FR11	The system shall allow customers to view upcoming and past service appointments.
FR12	The system shall allow customers to cancel or reschedule bookings before the appointment time.

Table 2.4: Functional Requirements (AI Customer Assistant and Recommendation Engine)

Req. ID	Description
FR13	The system shall allow users to input vehicle data including mileage, fuel efficiency, vehicle age, and component conditions.
FR14	The system shall process the input data using a Random Forest machine learning model.
FR15	The system shall generate predictions indicating whether maintenance is required.
FR16	The system shall identify the most relevant service type based on the prediction results.
FR17	The system shall display recommended services along with estimated urgency levels.
FR18	The system shall provide an AI chatbot interface to answer user queries related to vehicle maintenance and services.

Table 2.5: Functional Requirements (Multi-Branch Service and Inventory Management)

Req. ID	Description
FR19	The system shall allow managers to create and update branch profiles including branch location and contact information.
FR20	The system shall allow managers to define services offered at each branch along with service pricing.

FR21	The system shall track spare parts inventory levels across branches.
FR22	The system shall generate automated alerts when spare parts inventory falls below predefined thresholds.
FR23	The system shall allow managers to update stock levels and manage inventory records.

Table 2.6: Functional Requirements (Vehicle Service Tracking and Task Allocation)

Req. ID	Description
FR24	The system shall maintain service history records for each registered vehicle.
FR25	The system shall allow managers to view the workload and availability of mechanics.
FR26	The system shall assign service tasks to mechanics based on availability and expertise.
FR27	The system shall allow managers to manually reassign tasks when required.
FR28	The system shall update the service status throughout the maintenance process.
FR29	The system shall allow customers to view real-time updates regarding their service progress.

Table 2.7: Functional Requirements (Centralized Analytics and Dashboard System)

Req. ID	Description
FR30	The system shall generate real-time analytics for bookings, revenue, and inventory.
FR31	The system shall display branch-wise performance metrics through interactive dashboards.
FR32	The system shall provide graphical visualizations including charts and trend reports.

FR33	The system shall allow administrators to compare performance between branches.
FR34	The system shall support exporting analytical reports in PDF or Excel format.

Table 2.8: Functional Requirements (Notification and Activity Monitoring System)

Req. ID	Description
FR35	The system shall send booking confirmations through email or SMS.
FR36	The system shall notify customers about service progress updates.
FR37	The system shall send low-stock alerts to managers when inventory levels drop.
FR38	The system shall maintain activity logs for system actions.
FR39	The system shall allow users to configure notification preferences.

Table 2.9: Functional Requirements (Customer Feedback and Blog Management)

Req. ID	Description
FR40	The system shall allow customers to rate completed services.
FR41	The system shall allow customers to submit written feedback after service completion.
FR42	The system shall display testimonials and reviews on the platform.
FR43	The system shall allow administrators to publish blogs related to vehicle care and maintenance.

2.3.2 Behavioral Requirements

Behavioral requirements describe how users interact with the system through different tasks and workflows. These interactions are represented using use cases that define the roles of system actors and the sequence of actions performed.

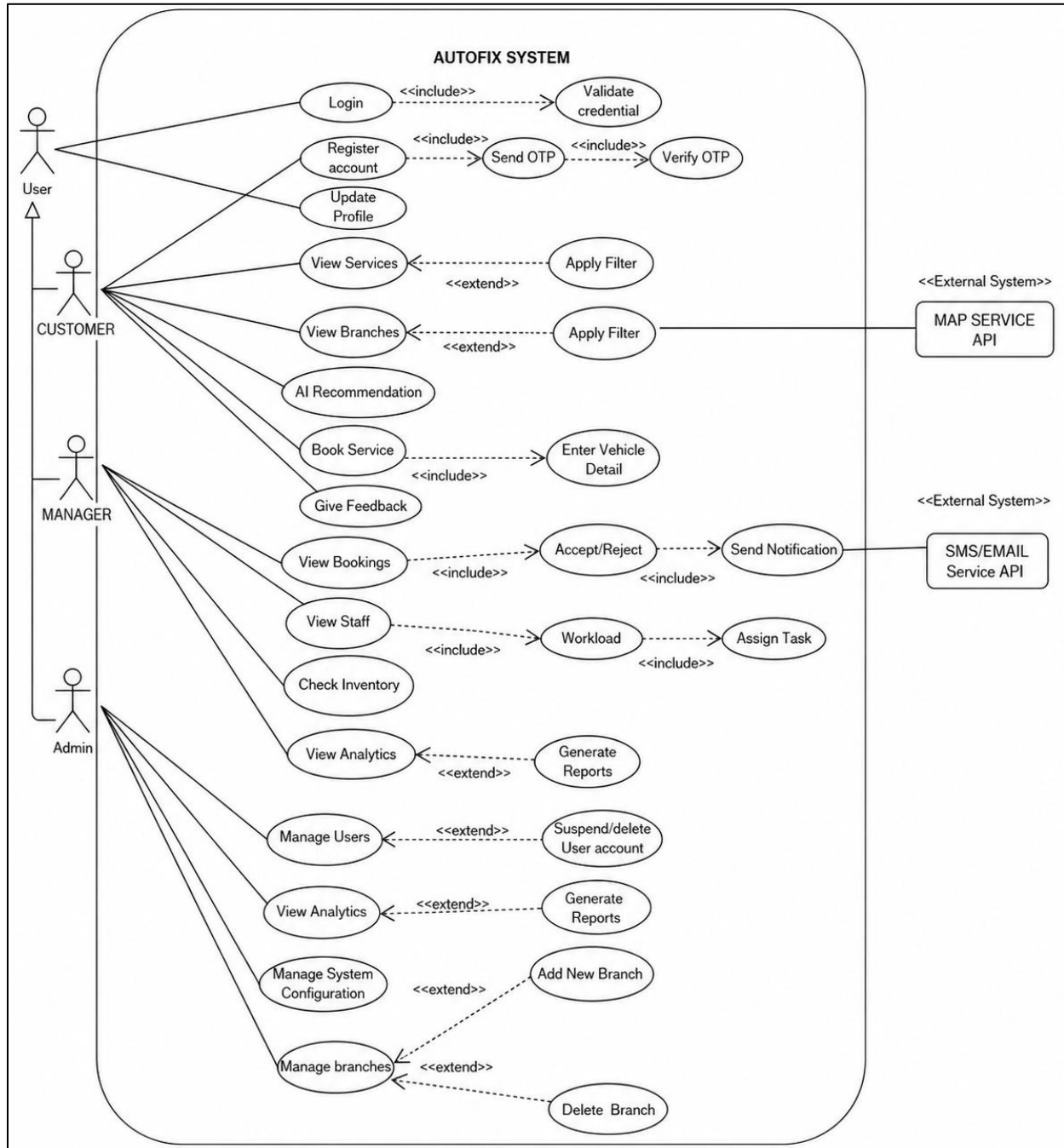


Figure 2.2: Use Case Diagram

2.3.3 External Interface Requirements

2.3.3.1 User Interface

The system interface will provide role-based dashboards for Admin, Branch Manager, and Customer, each designed with organized sections, intuitive controls, and attractive data visualization to enhance the overall user experience.



Figure 2.3: Admin Dashboard Overview

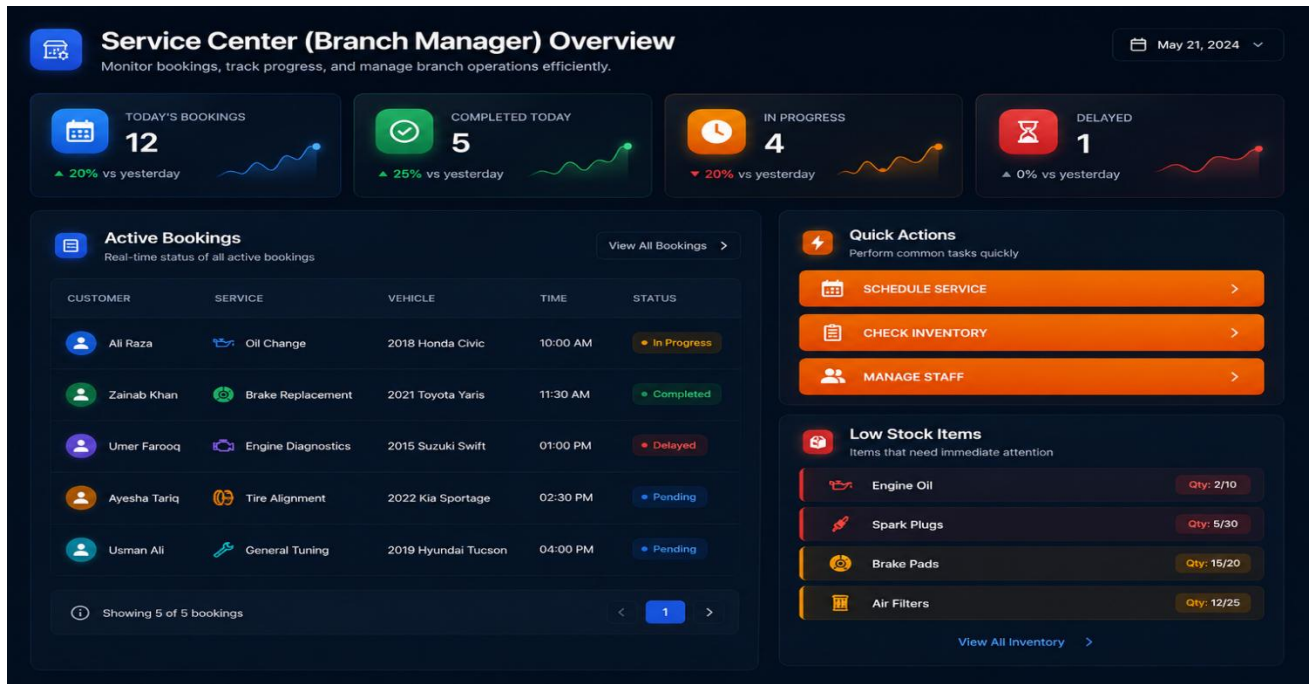


Figure 2.4: Branch Manager Dashboard overview

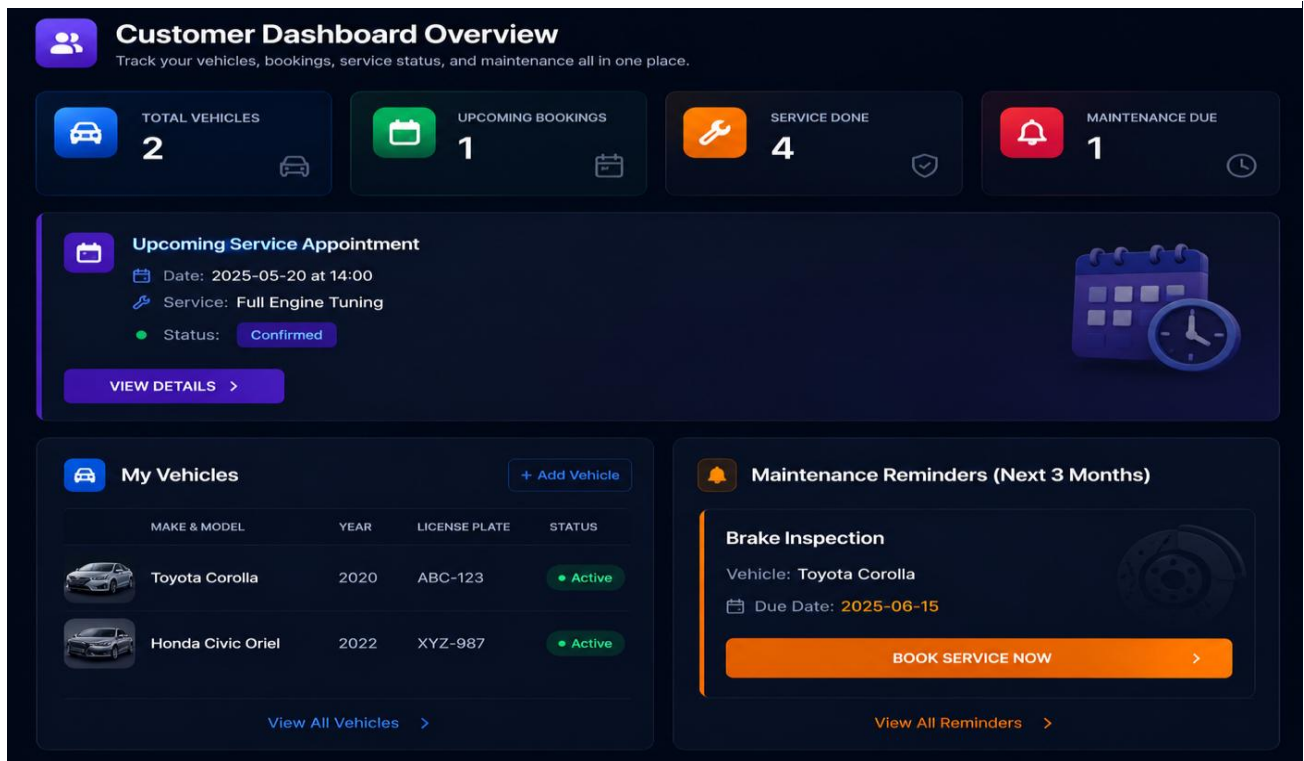


Figure 2.4: Customer Dashboard overview

▪ **Communication Interfaces**

- HTTPS-based REST APIs for all client-server communication
- HTTP/HTTPS protocol for web communication
- SMTP email services for booking confirmations and notifications
- SMS gateway services for alerts
- JSON-based data exchange between frontend and backend
- JWT-based encrypted token authentication for sensitive data

2.3.3.2 Other Interfaces

▪ **Hardware Interfaces:**

- AutoFix does not require specialized hardware
- The system runs on standard computing devices such as desktops, laptops, tablets
- Supports keyboard and mouse as input devices
- Uses display screens for visual interface rendering
- Requires internet-enabled devices to access the web platform
- Supports optional device cameras for uploading vehicle images when required

▪ **Software Interfaces:**

- AutoFix integrates with multiple software components and frameworks to support system functionality
- The system operates on Windows 10+, Linux (Ubuntu 18.04+), macOS 10.14
- Uses MySQL for structured data storage
- Uses Django REST Framework for backend API development
- Uses scikit-learn Random Forest classifier for AI-based predictions
- Uses Chart.js for analytics and data visualization

▪ **Communication Interfaces:**

- All communication between client interface and server is conducted through HTTPS-based REST APIs
- Supports HTTP/HTTPS protocols for web communication
- Uses SMTP email services for booking confirmations and notifications

- Uses SMS gateway services for sending alerts
- Uses JSON format for data exchange between frontend and backend APIs
- Sensitive information such as authentication tokens and personal data is transmitted through encrypted connections
- Uses JWT-based tokens for secure authentication and data protection

2.4 Non-functional Requirements

2.4.1 Performance Requirements

- **Response Time:** User requests processed within 3–5 seconds under normal network conditions
- **Transaction Processing Time:** Booking transactions completed within 10 seconds from submission to confirmation
- **Concurrent User Support:** Tested for 50 concurrent users without noticeable performance degradation
- **Database Query Efficiency:** Data retrieval operations execute within 2 seconds
- **AI Recommendation Processing:** Predictions generated within 5 seconds after receiving vehicle parameters
- **System Availability:** 99% uptime to ensure continuous user access
- **Scalability:** Architecture is designed to support up to 200+ concurrent users with future implementation of Redis as a message broker and asynchronous task processing

2.4.2 Safety and Security Requirements

- **Data Protection:** Sensitive user data encrypted; passwords stored using secure hashing algorithms (not plain text)
- **User Authentication:** Secure login credentials required; role-based authentication implemented
- **Access Control:** Customers limited to own data; managers limited to assigned branch; administrators have full access

- **Data Backup and Recovery:** Regular automated database backups; secure and recoverable copies
- **Input Validation:** All inputs validated to prevent SQL injection, XSS, and unauthorized manipulation
- **Secure Communication:** All communication over HTTPS encryption

2.4.3 Software Quality Attributes

- **Reliability:**
 - System recovers quickly from unexpected failures
 - Automated error logging implemented to identify system issues
 - Database transactions ensure data consistency and integrity
 - Regular system monitoring detects performance issues
- **Usability:**
 - Interface follows consistent layout and navigation patterns
 - Common tasks such as booking completable in a few simple steps
 - Help messages and prompts guide users when performing actions
 - Responsive and accessible from desktops, tablets, and mobile devices
- **Maintainability:**
 - Modular architecture separating frontend, backend, and AI modules
 - Clear code documentation maintained for future developers
 - Widely supported frameworks and libraries simplify maintenance
 - Codebase follows standard coding practices for easier maintenance
- **Portability:**
 - Accessible via Chrome, Firefox, Safari, and Edge
 - Backend runs on Windows, Linux, and macOS
 - Supports cloud server deployment for scalability

- Supports Docker for easy cross-environment deployment
- Deployable on multiple cloud platforms for flexibility

- **Scalability:**
 - Additional branches, users, and services can be added without affecting functionality
 - Database structure supports large volumes of records

- **Robustness:**
 - All input data validated before processing
 - System prevents crashes when invalid or incomplete data is provided
 - Error handling mechanisms ensure system stability

- **Testability:**
 - Application designed in modular components testable independently
 - Unit testing, integration testing, and system testing supported
 - Logging mechanisms assist developers in identifying defects during testing

3 DESIGN SPECIFICATIONS

This chapter presents the design specifications of the AutoFix system. It covers multiple viewpoints including the composite, logical, information, interaction, state dynamics, and algorithmic viewpoints. Each viewpoint provides a different perspective on the system design.

3.1 Introduction

This chapter describes the architectural and detailed design of the AutoFix vehicle service management system. It presents the system design through six distinct viewpoints, each supported by relevant UML diagrams and design models. The chapter provides a comprehensive understanding of how the system components are organized, how they interact, and how data flows through the platform.

3.2 Composite Viewpoint

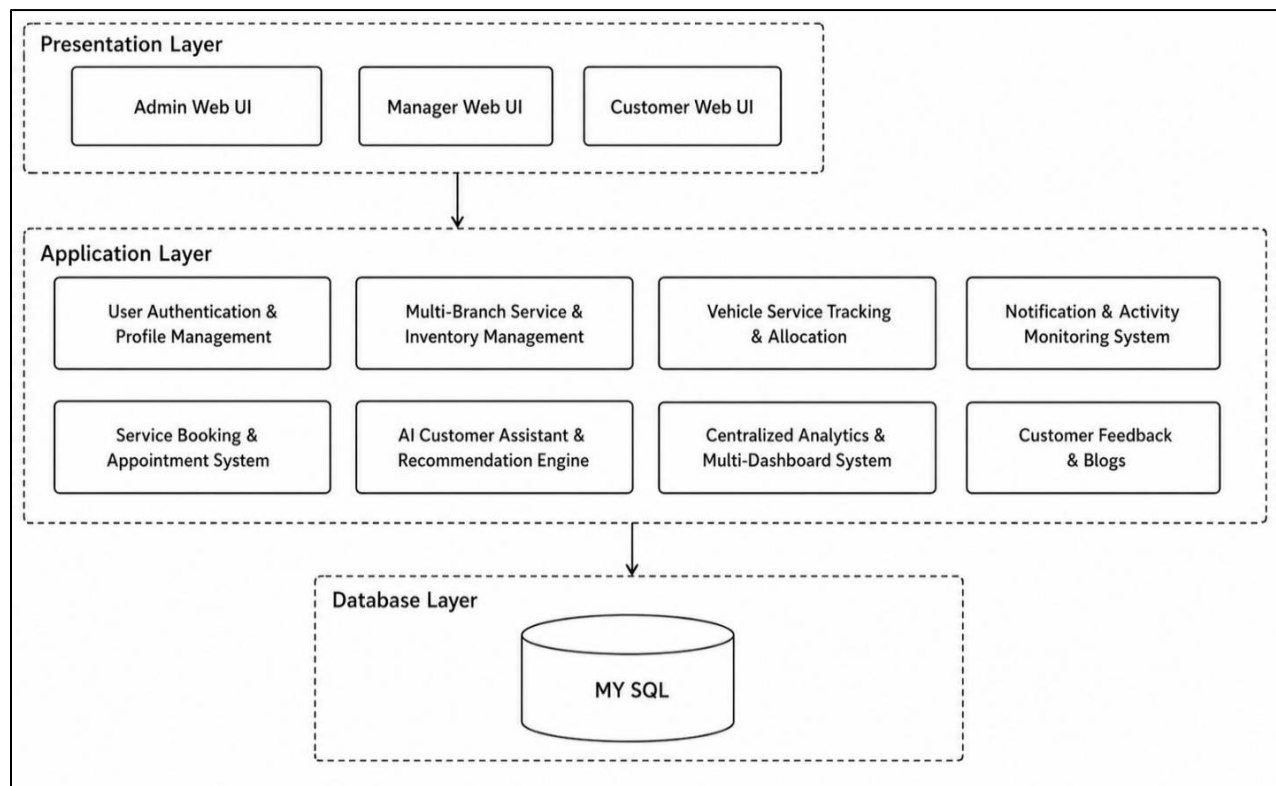


Figure 3.1: Package Diagram

3.3 Logical Viewpoint

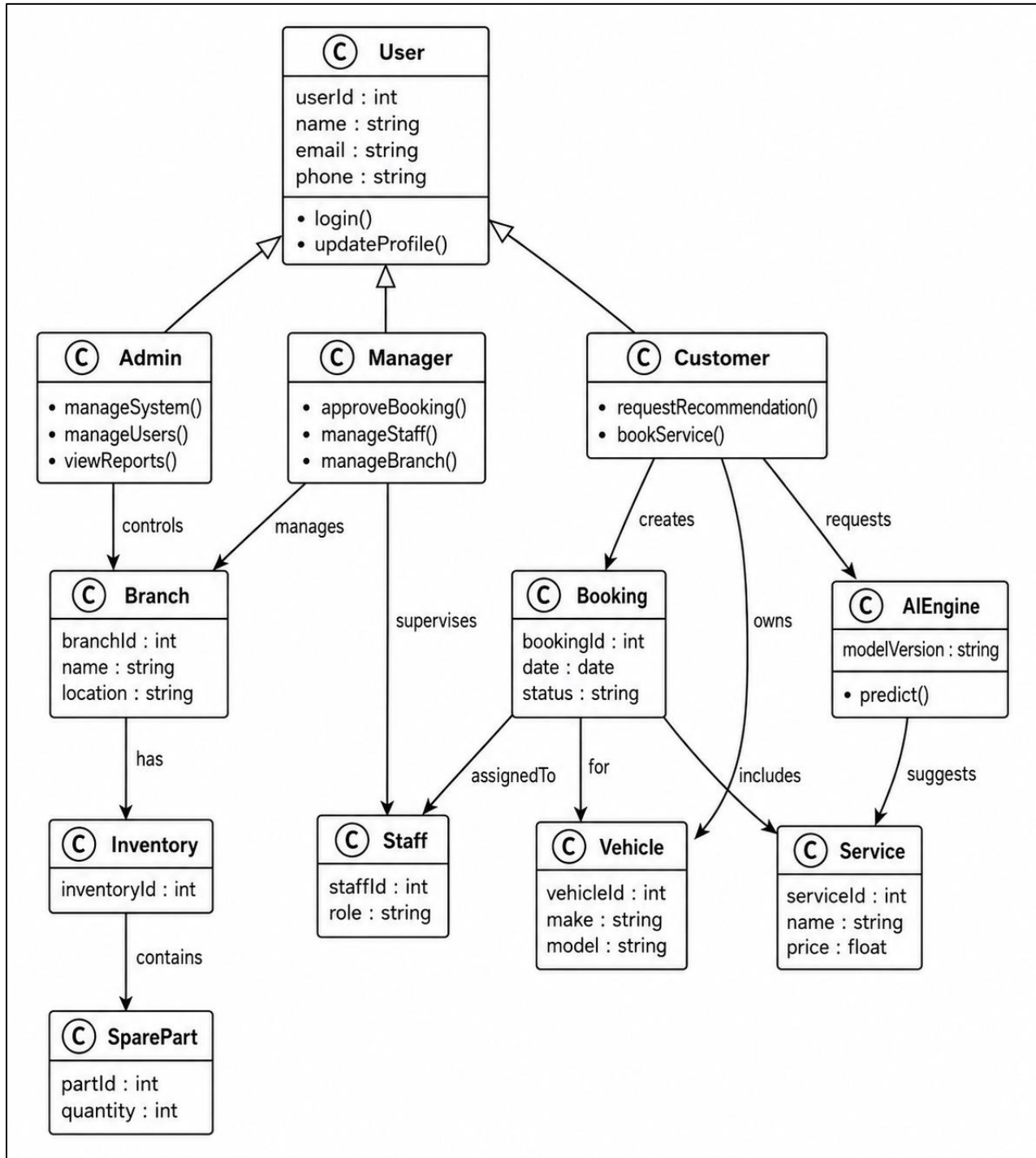


Figure 3.2: Class Diagram

3.4 Information Viewpoint

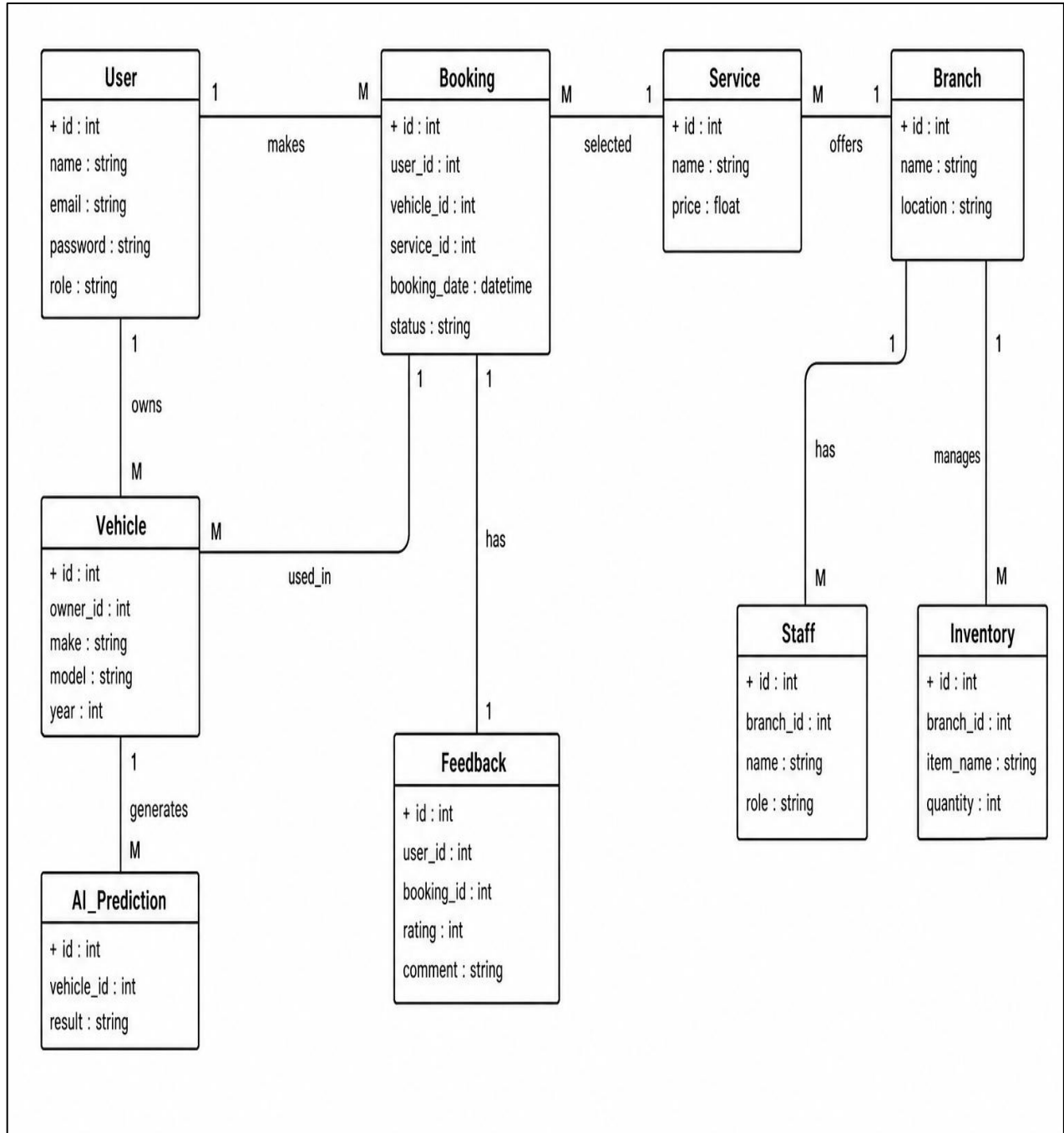


Figure 3.3: ERD Diagram

3.5 Interaction Viewpoint

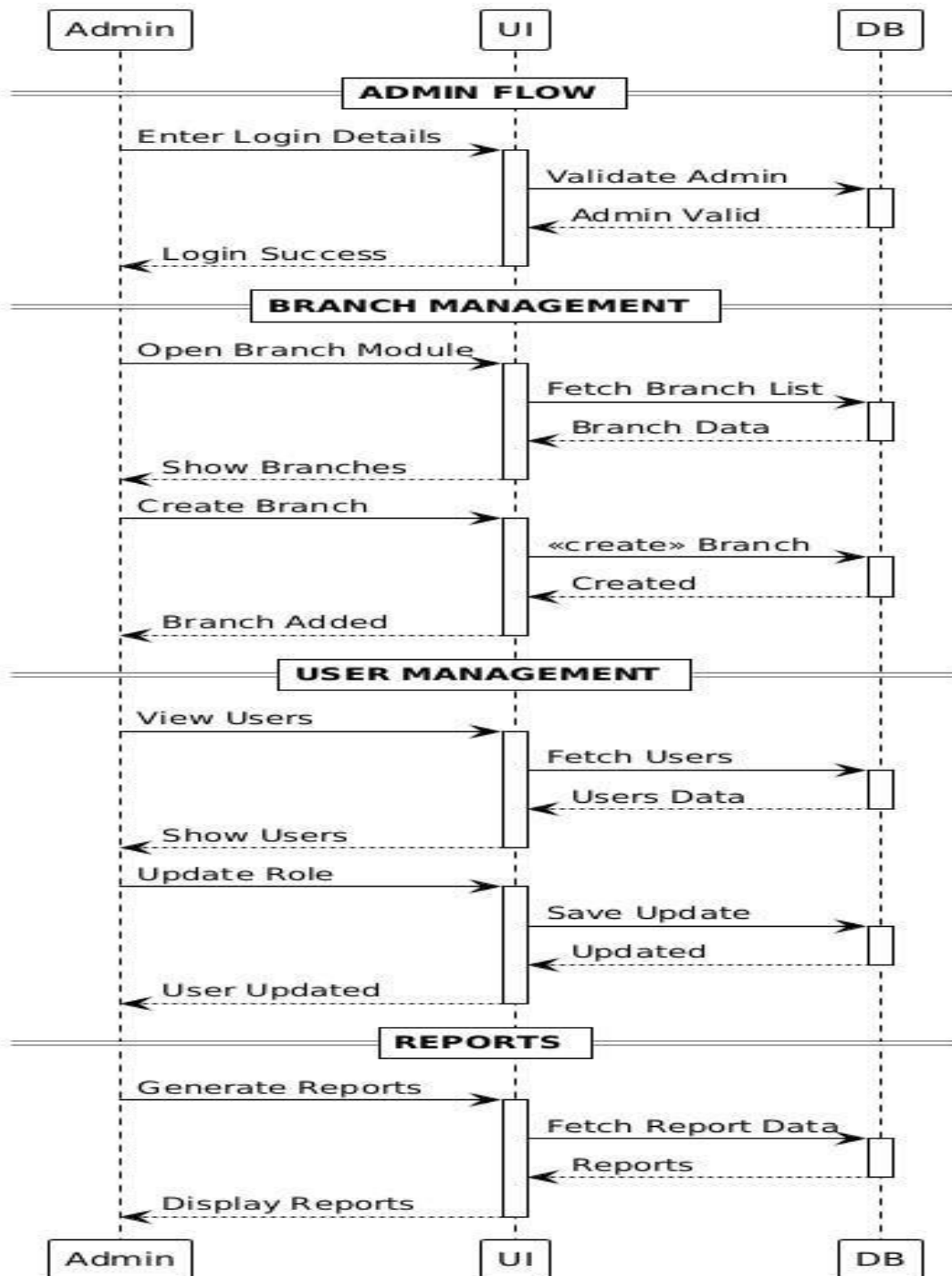


Figure 3.4: Admin Sequence Diagram

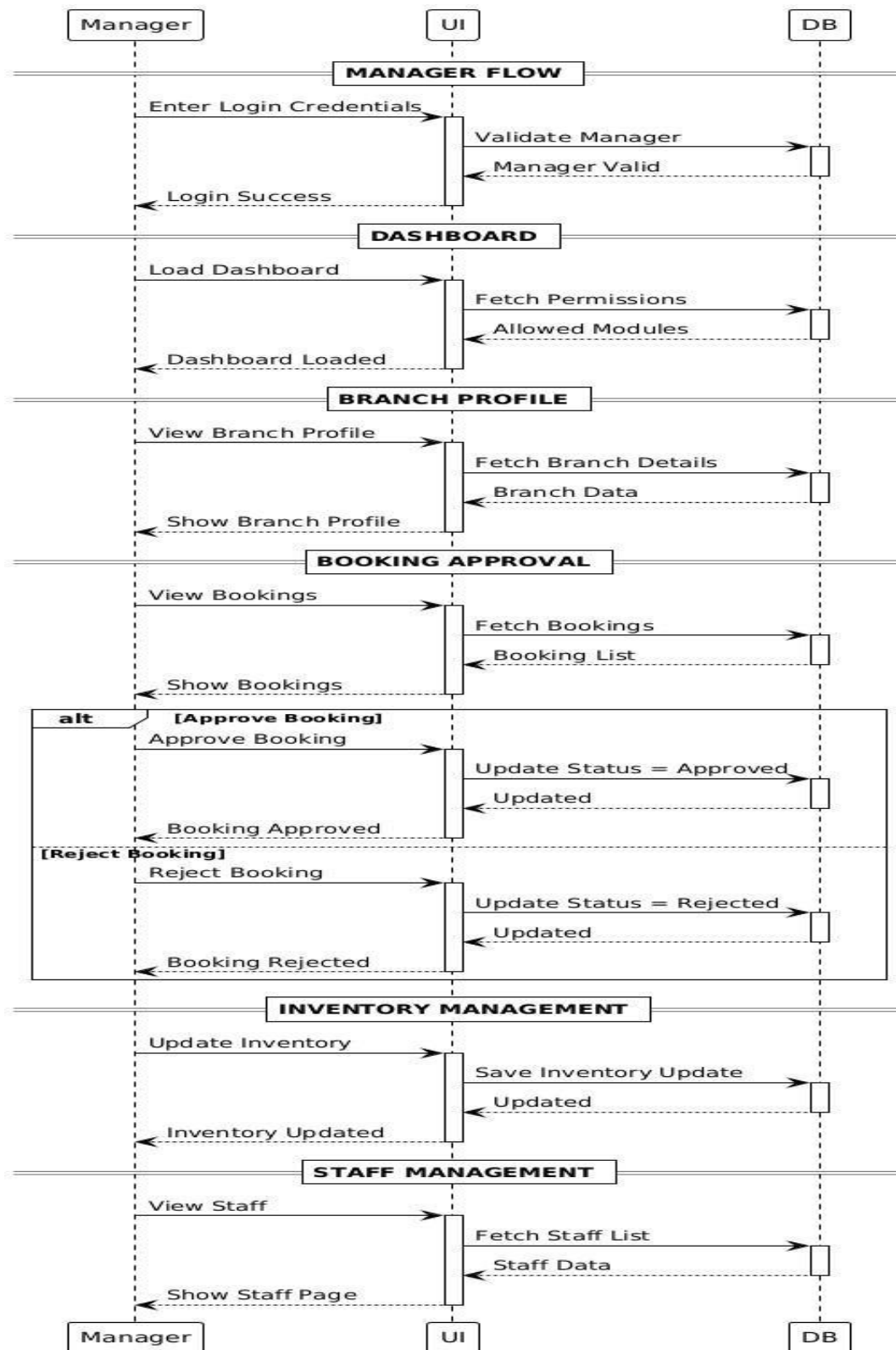


Figure 3.5: Manager Sequence Diagram

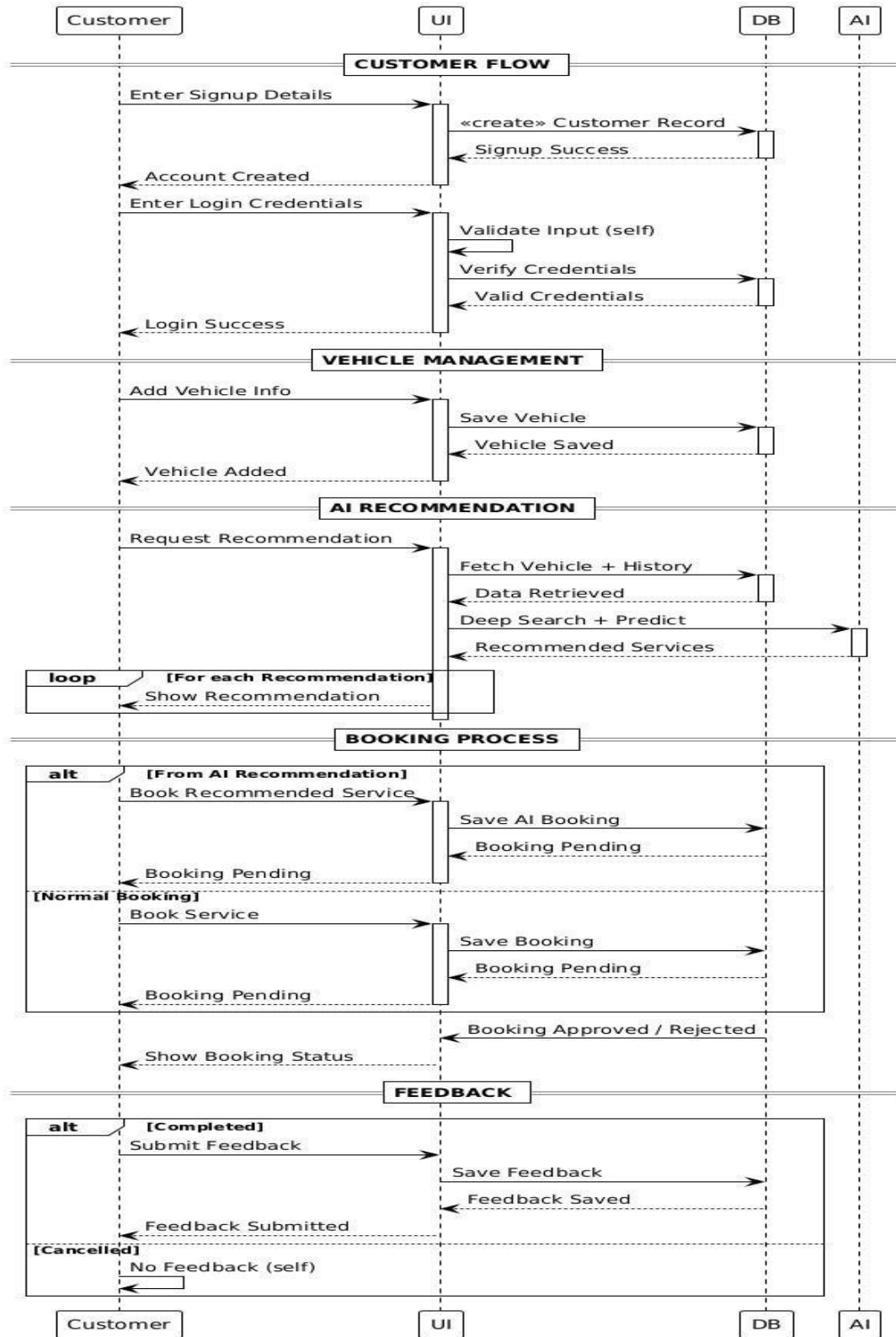


Figure 3.6: Customer Sequence Diagram

3.6 State Dynamics Viewpoint

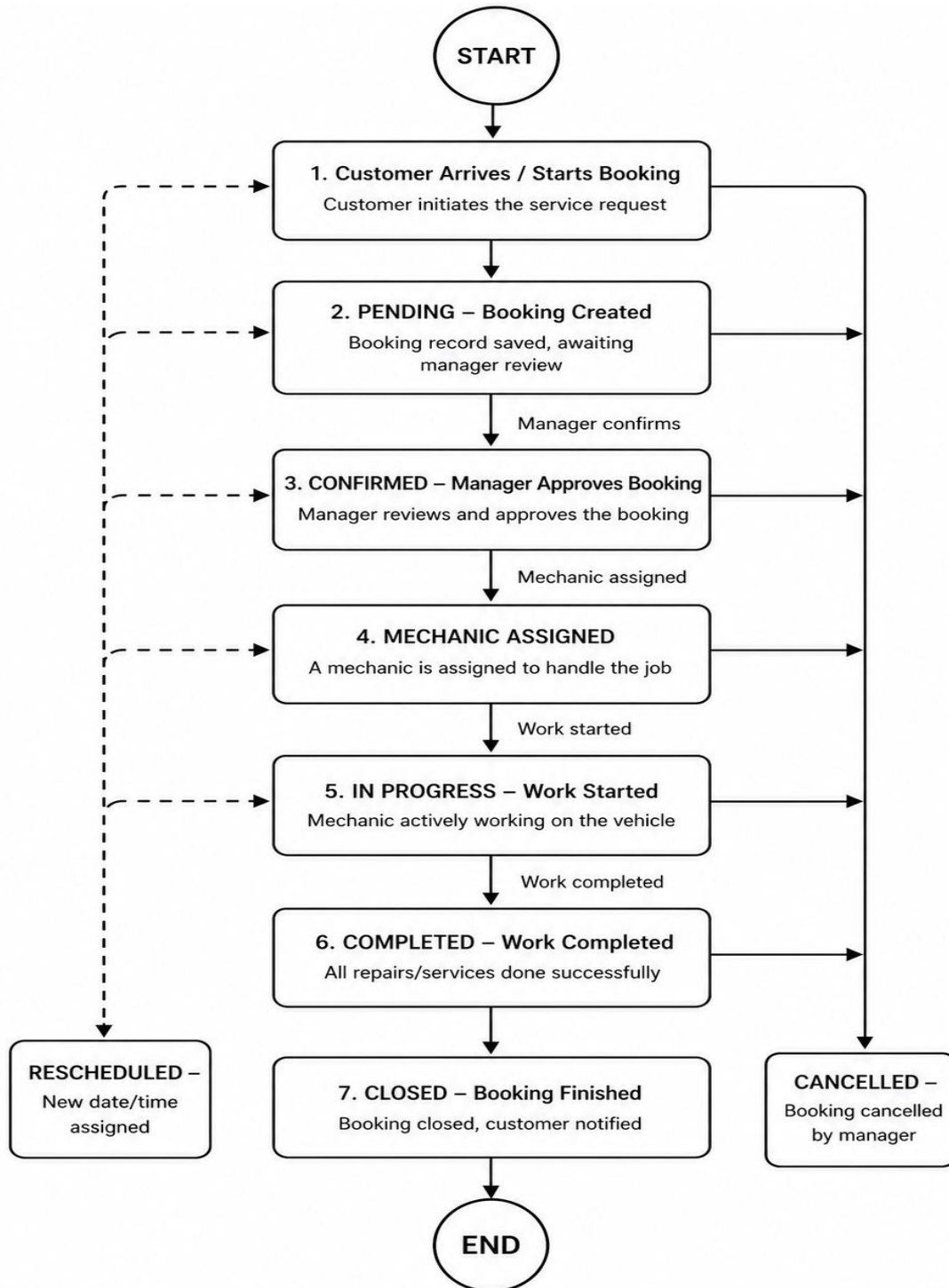


Figure 3.7: Booking Process State Diagram

3.7 Algorithmic Viewpoint

3.7.1 AI Maintenance Prediction Algorithm

```
ALGORITHM PredictMaintenance(vehicle_data)
INPUT: mileage, fuel_efficiency, condition, age
OUTPUT: {maintenance_type, urgency, confidence}

BEGIN
  model <- LOAD('random_forest_model.pkl')
  features <- [
    NORMALIZE(vehicle_data.mileage),
    NORMALIZE(vehicle_data.fuel_efficiency),
    ENCODE(vehicle_data.condition),
    NORMALIZE(vehicle_data.age)
  ]
  predictions <- []
  FOR each tree IN model.trees DO
    predicted_class <- tree.predict(features)
    predictions.append(predicted_class)
  END FOR
  maintenance_type <- MAJORITY_VOTE(predictions)
  confidence <- COUNT(predictions == maintenance_type) / TOTAL_TREES
  urgency <- GET_URGENCY(vehicle_data, confidence)
  RETURN { maintenance_type, urgency, confidence }
END
```

3.7.2 Booking Creation Algorithm

```
ALGORITHM CreateBooking(customer_id, vehicle_id, booking_data)
INPUT: customer_id, vehicle_id, date, branch, time_slot
OUTPUT: booking_confirmation
BEGIN
```

```

VERIFY customer exists
VERIFY vehicle exists AND belongs to customer
vehicle_data <- GET_VEHICLE_DATA(vehicle_id)
prediction <- PredictMaintenance(vehicle_data)
estimate <- EstimateCostAndTime(prediction.maintenance_type, vehicle_data.age)
booking_id <- GENERATE_ID()
booking <- {
  id: booking_id,
  customer_id: customer_id,
  vehicle_id: vehicle_id,
  branch_id: booking_data.branch,
  service_type: prediction.maintenance_type,
  status: 'PENDING' }
DATABASE.INSERT('bookings', booking)
SEND_NOTIFICATION('New booking created', customer_id)
booking_confirmation <- {
  booking_id: booking_id,
  service_type: prediction.maintenance_type,
  estimated_cost: estimate.cost, estimated_time: estimate.duration_minutes,
  estimated_time: estimate.duration_minutes,
  status: 'PENDING' }
RETURN booking_confirmation
END

```

4 DEVELOPMENT AND TOOLS

This chapter describes the development approach, tools, and technologies used in building the AutoFix system. It provides an overview of the development plan, including team responsibilities and task distribution. Additionally, it highlights the software tools, frameworks, and technologies used during implementation.

4.1 Introduction

This chapter presents the development methodology and tools used in the implementation of AutoFix. It includes the development plan outlining team members and task distribution, followed by a detailed description of the tools and technologies used in both frontend and backend development.

4.2 Development Plan

- **Team Members:**
 - Abdul Wasay — CIIT/FA22-BCS-127/WAH
 - Ahmad Ali — CIIT/FA22-BCS-144/WAH
- **Abdul Wasay Responsibilities:**
 - Service Booking & Appointment System
 - AI Customer Assistant & Recommendation Engine
 - Centralized Analytics & Multi-Dashboard System
 - Customer Feedback & Blogs Module
 - Frontend UI design (React.js components for assigned modules)
- **Ahmad Ali Responsibilities:**
 - User Authentication & Profile Management
 - Multi-Branch Service & Inventory Management
 - Vehicle Service Tracking & Allocation System
 - Notification & Activity Monitoring System
 - Backend API development (Django REST Framework)

4.2.1 Development Phases:

Table 4.1: Development Phases

Phase	Activity / Description
Phase 1	Requirement Analysis — Gathering system requirements and defining modules
Phase 2	System Design — Designing architecture, database schema, and UI wireframes
Phase 3	Frontend Development — Developing UI using React.js
Phase 4	Backend Development — API development using Django REST Framework
Phase 5	AI Model Integration — Training and integrating Random Forest model
Phase 6	Testing — Unit testing, integration testing, and bug fixing
Phase 7	Deployment — Hosting system and final deployment

4.2.2 Development Timeline (Gantt Chart):

Table 11: Development Timeline — Gantt Chart

ID	Task / Title	Start Date	End Date	Responsible
1.1	Planning	Oct 20, 2025	Oct 28, 2025	Ahmad, Abdul Wasay
1.1.1	Idea Finalization & Problem Statement	Oct 20, 2025	Oct 22, 2025	Ahmad, Abdul Wasay
1.1.2	Project Scope Definition	Oct 23, 2025	Oct 25, 2025	Ahmad Ali
1.1.3	Documentation Plan	Oct 26, 2025	Oct 28, 2025	Abdul Wasay
1.2	Requirement Gathering	Oct 29, 2025	Nov 10, 2025	Ahmad, Abdul Wasay
1.2.1	Basic Requirement Draft (SRS)	Oct 29, 2025	Nov 03, 2025	Abdul Wasay

1.2.2	Requirement Specification & Review	Nov 04, 2025	Nov 10, 2025	Ahmad Ali
1.3	Analysis	Nov 11, 2025	Nov 20, 2025	Ahmad, Abdul Wasay
1.3.1	Requirement Analysis & Feasibility	Nov 11, 2025	Nov 16, 2025	Ahmad Ali
1.3.2	Requirement Verifiability	Nov 17, 2025	Nov 20, 2025	Abdul Wasay
1.4	Design	Nov 21, 2025	Dec 10, 2025	Ahmad, Abdul Wasay
1.4.1	Basic UML Diagram	Nov 21, 2025	Nov 27, 2025	Abdul Wasay
1.4.2	UML Diagrams (Use Case, Class, Sequence)	Nov 28, 2025	Dec 04, 2025	Ahmad, Abdul Wasay
1.4.3	Architecture Design & SDD Document	Dec 05, 2025	Dec 10, 2025	Ahmad Ali
1.5	Implementation	Dec 11, 2025	Mar 20, 2026	Ahmad, Abdul Wasay
1.5.1	30% Dev: Auth, Login/Signup, User Profiles	Dec 11, 2025	Jan 15, 2026	Ahmad, Abdul Wasay
1.5.2	70% Dev: AI Booking, Predictive Maintenance, Inventory Module	Jan 16, 2026	Feb 20, 2026	Ahmad, Abdul Wasay
1.5.3	85% Dev: Analytics, Admin Panel, Branch Management	Feb 21, 2026	Mar 20, 2026	Ahmad, Abdul Wasay
1.6	Testing	Mar 21, 2026	Apr 05, 2026	Ahmad, Abdul Wasay
1.6.1	Unit & Integration Testing	Mar 21, 2026	Mar 28, 2026	Ahmad Ali
1.6.2	System & User Acceptance Testing	Mar 29, 2026	Apr 05, 2026	Abdul Wasay
1.7	Final Report	Apr 06, 2026	Apr 25, 2026	Ahmad, Abdul Wasay

1.7.1	Basic Draft	Apr 06, 2026	Apr 15, 2026	Abdul Wasay
1.7.2	Final Draft & Supervisor Review	Apr 16, 2026	Apr 25, 2026	Ahmad Ali
1.8	Deployment	Apr 26, 2026	May 08, 2026	Ahmad, Abdul Wasay
1.8.1	Final Build & Testing	Apr 26, 2026	May 05, 2026	Ahmad, Abdul Wasay
1.8.2	Final Release & Submission	May 06, 2026	May 08, 2026	Ahmad, Abdul Wasay

4.3 Development Tools

- **Frontend Development Tools:**

- **React.js:** For building dynamic and responsive user interfaces
- **HTML5 & CSS3:** For structuring and styling web pages
- **JavaScript (ES6+):** For frontend logic and interactivity
- **Chart.js:** For graphical data visualization in dashboards
- **Bootstrap / Tailwind CSS:** For responsive UI design

- **Backend Development Tools:**

- **Python:** Core programming language
- **Django REST Framework:** For building RESTful APIs
- **JWT (JSON Web Tokens):** For secure authentication system

- **Database Tools:**

- **MySQL:** Relational database for storing system data
- **Redis:** For caching and improving system performance

- **AI and Machine Learning Tools:**

- **Scikit-learn:** For implementing Random Forest classifier

- **Pandas & NumPy:** For data preprocessing and analysis
- **Development Environment (IDE & Tools):**
 - **Visual Studio Code (VS Code):** Primary code editor
 - **Postman:** API testing and debugging
 - **Git & GitHub:** Version control system
 - **Figma:** UI/UX design and prototyping
- **Deployment Tools:**
 - **Docker:** For containerization
 - **Cloud Hosting (AWS / VERCEL):** For deployment
 - **Nginx:** Web server for production deployment

4.4 Conclusion and Future Work/Extensions

The system achieves its primary objectives of enhancing customer experience, improving operational efficiency, and enabling data-driven management for automotive service centers.

Planned Future Enhancements:

- Mobile application development for Android and iOS platforms
- Integration of IoT sensors for real-time vehicle diagnostics
- Advanced AI models using deep learning for more accurate predictions
- Online payment gateway integration for service payments
- Live chat support system for customer assistance
- GPS-based tracking of service centers and vehicle pickup/delivery services
- Cloud-based scaling for handling nationwide or international service networks

5 QUALITY ASSURANCE

This chapter describes the quality assurance activities performed to ensure that the AutoFix system meets its functional and non-functional requirements. It includes testing strategies, test cases, and a requirement traceability matrix. The purpose is to verify that all modules of the system are functioning correctly, efficiently, and reliably.

5.1 Introduction

This chapter focuses on the quality assurance process used in the development of the AutoFix system. It includes the testing approach adopted to ensure system reliability, correctness, and performance. The chapter presents a requirement traceability matrix that links functional requirements with test cases, ensuring that each requirement is properly validated.

5.2 Traceability Matrix

The Requirement Traceability Matrix (RTM) ensures that every functional requirement is mapped to one or more test cases, guaranteeing all system features are properly tested and validated.

Table 5.1: Traceability Matric for AutoFix

Req.ID	Description	TC_1	TC_2	TC_3	TC_4	TC_5	TC_6	TC_7	TC_8	Tests
Req_1	User Registration	Y								1
Req_2	User Login	Y	Y							2
Req_3	Profile Update		Y	Y						2
Req_4	Vehicle Management			Y	Y					2
Req_5	Service Booking	Y	Y	Y						3
Req_6	AI Recommendation				Y	Y				2
Req_7	Inventory Management					Y	Y			2

Req_8	Service Tracking						Y	Y		2
Req_9	Analytics Dashboard							Y	Y	2
Req_10	Feedback System								Y	2

5.3 Test Plan

The test plan defines the strategy used to verify each functional module of the AutoFix system. It ensures that all features work correctly under expected and edge conditions. Each test case includes input, expected output, and actual output validation.

Table 5.2: Test Case TC-1 (User Registration)

Test ID	TC-1
Test Name	User Registration
Date of Test	10/04/2026
Application	AutoFix
Description	This test verifies that a new user can register successfully using valid credentials.
Input	Name, Email, Phone, Password
Expected Output	User account created successfully and redirected to login page
Actual Output	User account created successfully
Test Role (Actor)	Customer
Verified By	Team Member

Table 5.3: Test Case TC-2 (User Login)

Test ID	TC-2
Test Name	User Login
Date of Test	10/04/2026
Application	AutoFix

Description	This test verifies that registered users can log in using valid credentials.
Input	Email, Password
Expected Output	User redirected to dashboard
Actual Output	User successfully logged in
Test Role (Actor)	Customer / Manager / Admin
Verified By	Team Member

Table 5.4: Test Case TC-3 (Service Booking)

Test ID	TC-3
Test Name	Service Booking
Date of Test	11/04/2026
Application	AutoFix
Description	This test verifies that customers can book a service successfully.
Input	Vehicle details, service type, branch, time slot
Expected Output	Booking confirmation generated
Actual Output	Booking successfully created
Test Role (Actor)	Customer
Verified By	Team Member

Table 5.5: Test Case TC-4 (AI Maintenance Prediction)

Test ID	TC-4
Test Name	AI Maintenance Prediction
Date of Test	11/04/2026
Application	AutoFix
Description	This test verifies that the AI model correctly predicts vehicle maintenance needs.
Input	Mileage, fuel efficiency, vehicle condition

Expected Output	Maintenance recommendation displayed
Actual Output	Correct prediction generated
Test Role (Actor)	Customer
Verified By	Team Member

Table 5.6: Test Case TC-5 (Inventory Management)

Test ID	TC-5
Test Name	Inventory Update
Date of Test	12/04/2026
Application	AutoFix
Description	This test ensures inventory can be updated and tracked correctly.
Input	Spare part quantity update
Expected Output	Inventory updated successfully
Actual Output	Inventory updated
Test Role (Actor)	Manager
Verified By	Team Member

Table 5.7: Test Case TC-6 (Service Tracking)

Test ID	TC-6
Test Name	Service Tracking
Date of Test	12/04/2026
Application	AutoFix
Description	This test verifies real-time service tracking functionality.
Input	Booking ID
Expected Output	Service status displayed
Actual Output	Status updated correctly
Test Role (Actor)	Customer / Manager

Verified By	Team Member
--------------------	-------------

Table 5.8: Test Case TC-7 (Analytics Dashboard)

Test ID	TC-7
Test Name	Analytics View
Date of Test	13/04/2026
Application	AutoFix
Description	This test checks if analytics data is displayed correctly.
Input	Dashboard request
Expected Output	Graphs and statistics displayed
Actual Output	Data shown correctly
Test Role (Actor)	Admin
Verified By	Team Member

Table 5.9: Test Case TC-8 (Feedback System)

Test ID	TC-8
Test Name	Submit Feedback
Date of Test	13/04/2026
Application	AutoFix
Description	This test verifies that users can submit feedback after service completion.
Input	Rating and comment
Expected Output	Feedback saved successfully
Actual Output	Feedback recorded
Test Role (Actor)	Customer
Verified By	Team Member

6 USER MANUAL

This chapter provides a complete guide for end users to understand how to install, configure, and operate the AutoFix system. It includes hardware and software requirements, step-by-step installation instructions, and a detailed operating manual.

6.1 Introduction

This chapter explains how to install and use the AutoFix system. It provides detailed instructions for system setup, required software and hardware specifications, and step-by-step operational guidance. The chapter is designed to assist both technical and non-technical users in successfully deploying and using the system.

6.2 Hardware/Software Requirements for the System

- **Hardware Requirements:**
 - **Processor:** Dual-Core 2.0 GHz or higher
 - **RAM:** Minimum 4 GB (8 GB recommended)
 - **Storage:** At least 100 MB free space
 - **Display:** Minimum resolution 1024 x 768
 - **Internet Connection:** Stable broadband or mobile data
 - **Input Devices:** Keyboard and mouse or touchscreen device
- **Software Requirements:**
 - **Operating System:** Windows 10+, macOS 10.14+, Android (via browser)
 - **Web Browsers:** Google Chrome (recommended), Mozilla Firefox, Microsoft Edge
 - **Backend:** Python 3.8+, Django Framework, Django REST Framework
 - **Database:** MySQL Server
 - **Frontend:** React.js, HTML5, CSS3, JavaScript

6.3 Installation guide for Application

Step 1: Clone the Project

Open terminal or command prompt and run:

```
git clone <repository-link>
```

Step 2: Install Backend Dependencies:

```
cd backend  
pip install -r requirements.txt
```

Step 3: Configure Database:

Install MySQL Server. Create a database named autofix_db. Update database credentials in settings.py.

Step 4: Run Database Migrations:

```
python manage.py makemigrations  
python manage.py migrate
```

Step 5: Start Backend Server:

```
python manage.py runserver  
# Backend runs at: http://127.0.0.1:8000/
```

Step 6: Install Frontend Dependencies:

```
cd frontend  
npm install
```

Step 7: Run Frontend Application:

```
npm start  
# Frontend runs at: http://localhost:3000/
```

Step 8: Access System:

Open the browser, enter the frontend URL (http://localhost:3000/), and login using registered credentials.

6.4 Operating Manual

This section explains how different users interact with the AutoFix system.

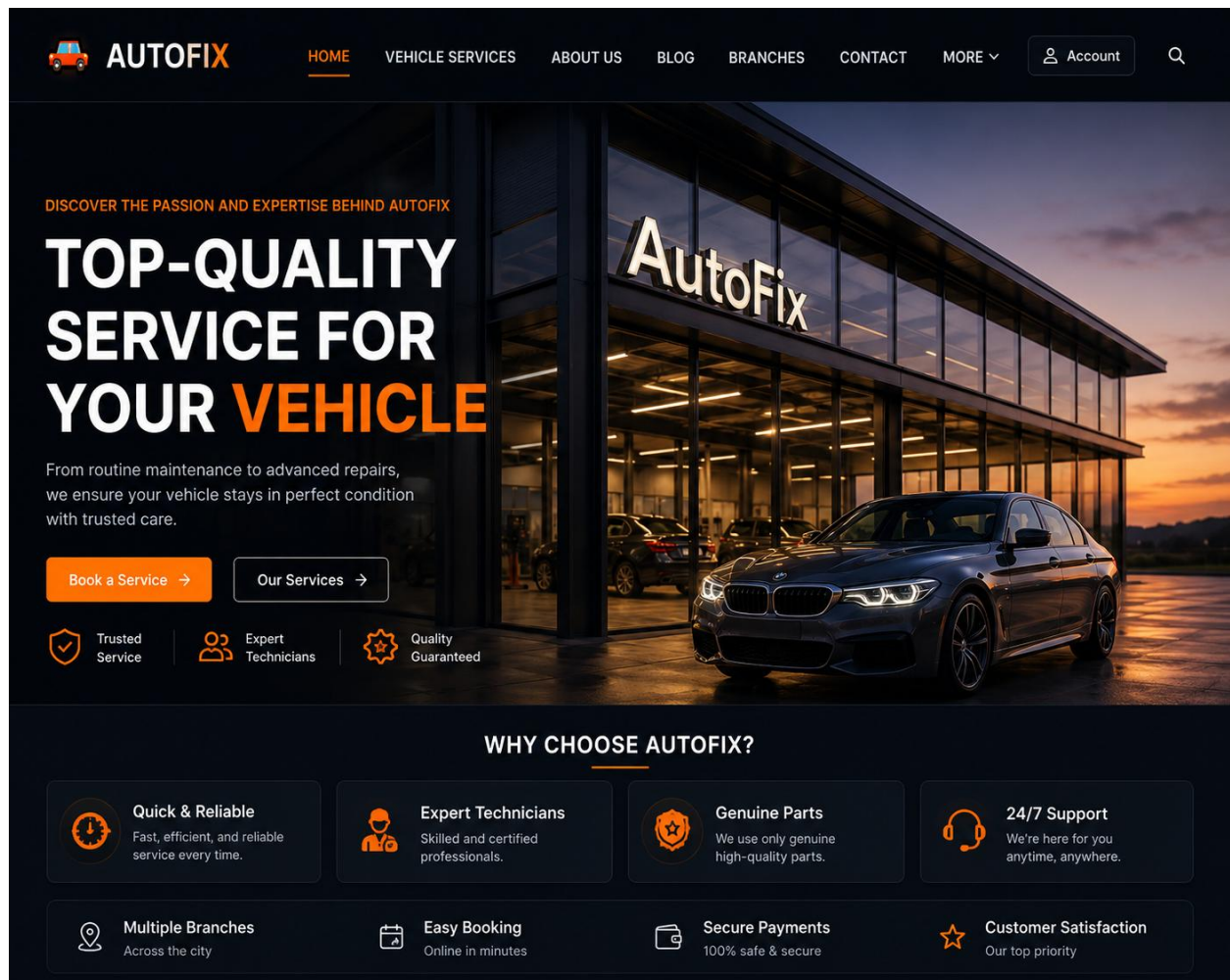


Figure 6.1: Home Page (Hero Section Screen)

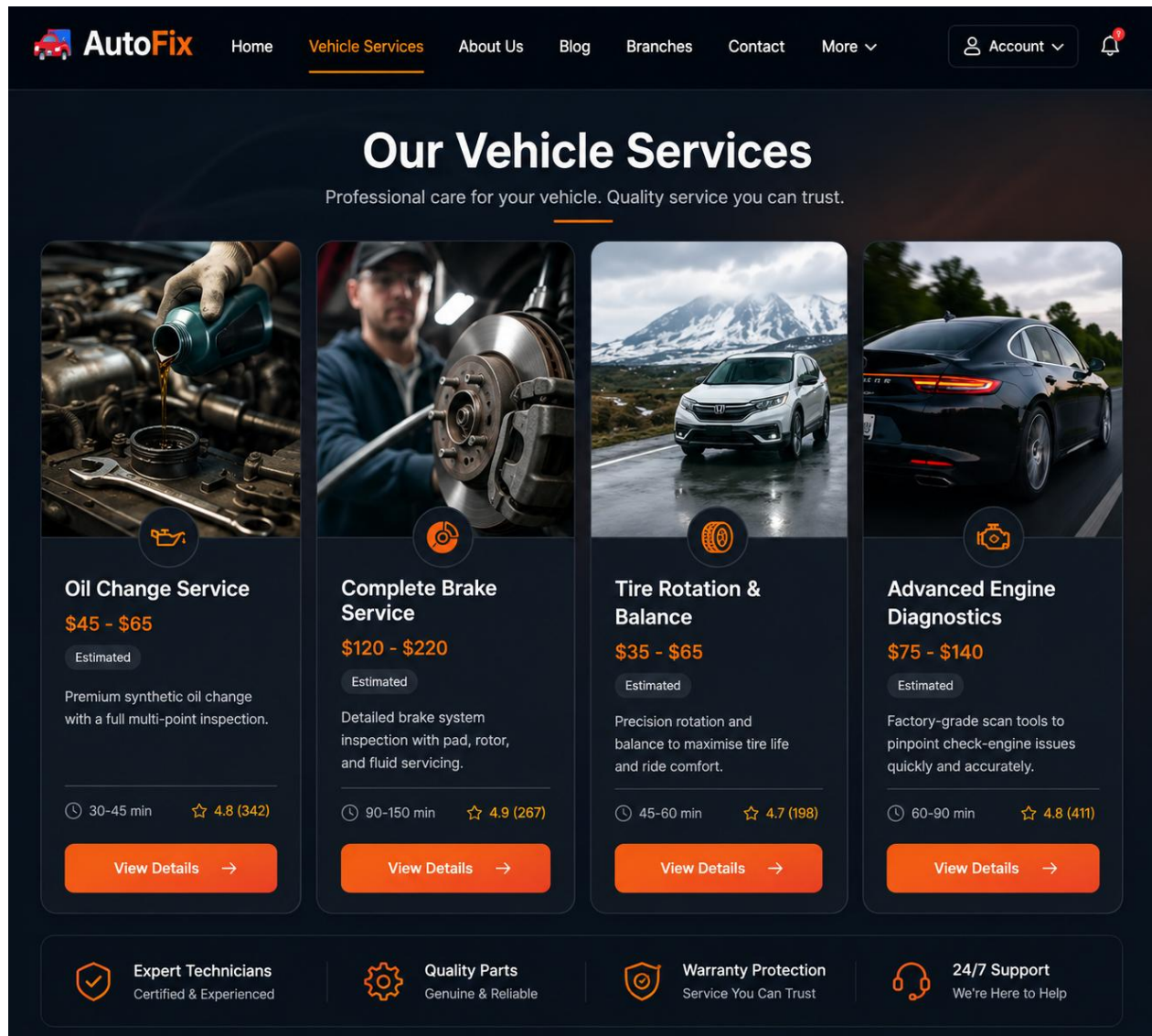


Figure 6.2: Vehicle Services Page Screen

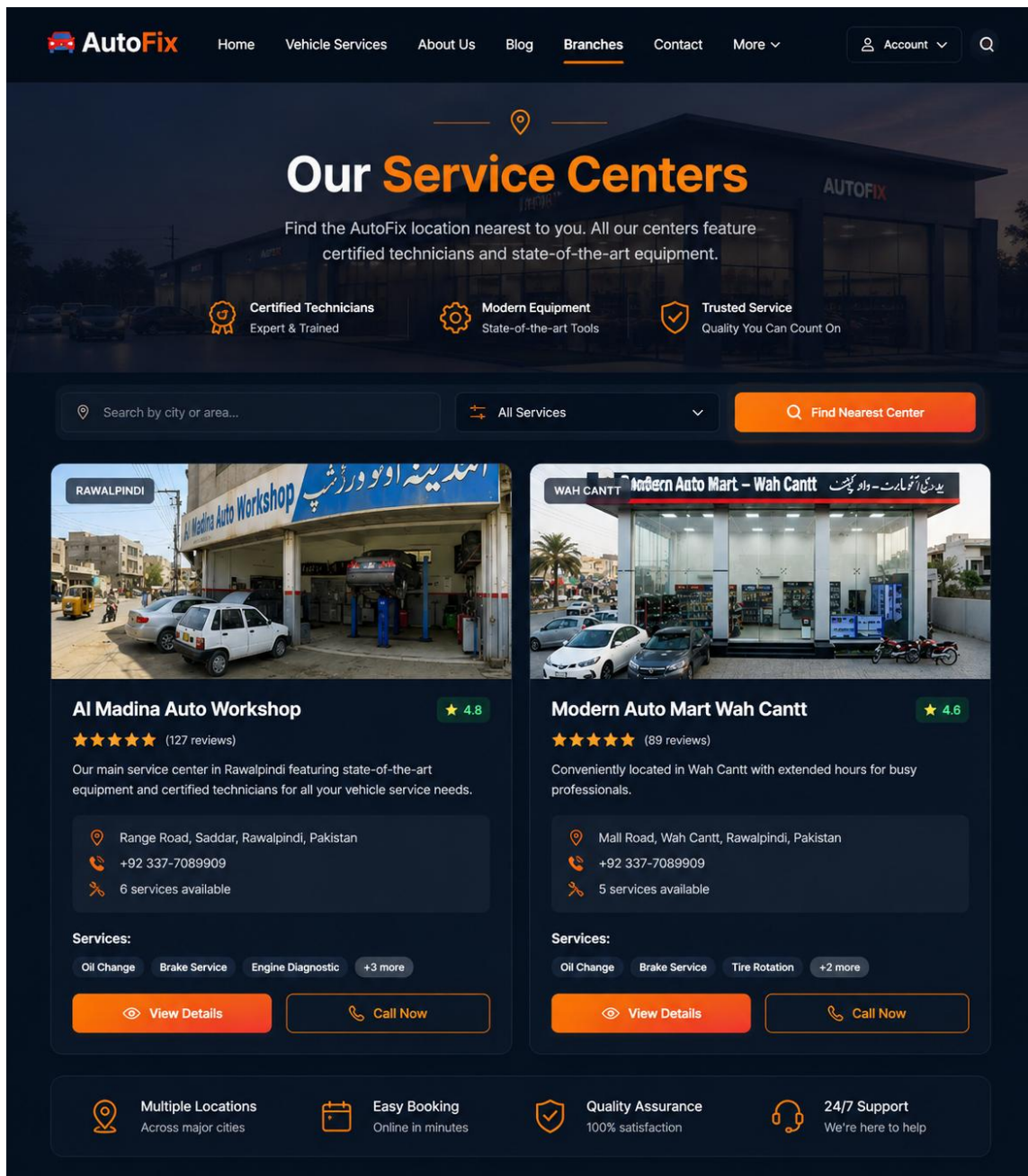


Figure 6.3: Branches / Service Centers Screen

**Service Details**

SERVICE TYPE *

 Select service type

URGENCY

 Normal

**Vehicle Information**

SELECT YOUR VEHICLE *

 Select from your vehicles

MAKE *

 Auto-filled from select

MODEL *

 Auto-filled from select

YEAR *

 Auto-filled from select

**Appointment Details**

PREFERRED DATE *

 dd/mm/yyyy

PREFERRED TIME *

 Select time slot

PREFERRED SERVICE CENTER

 Select preferred service center

SERVICE DESCRIPTION

 Describe the issue or service needed...

**Contact Information**

PHONE NUMBER *

 +1 (555) 123-4567

EMAIL *

 your@email.com

RESET

BOOK APPOINTMENT

Figure 6.4: Book Service Appointment Screen

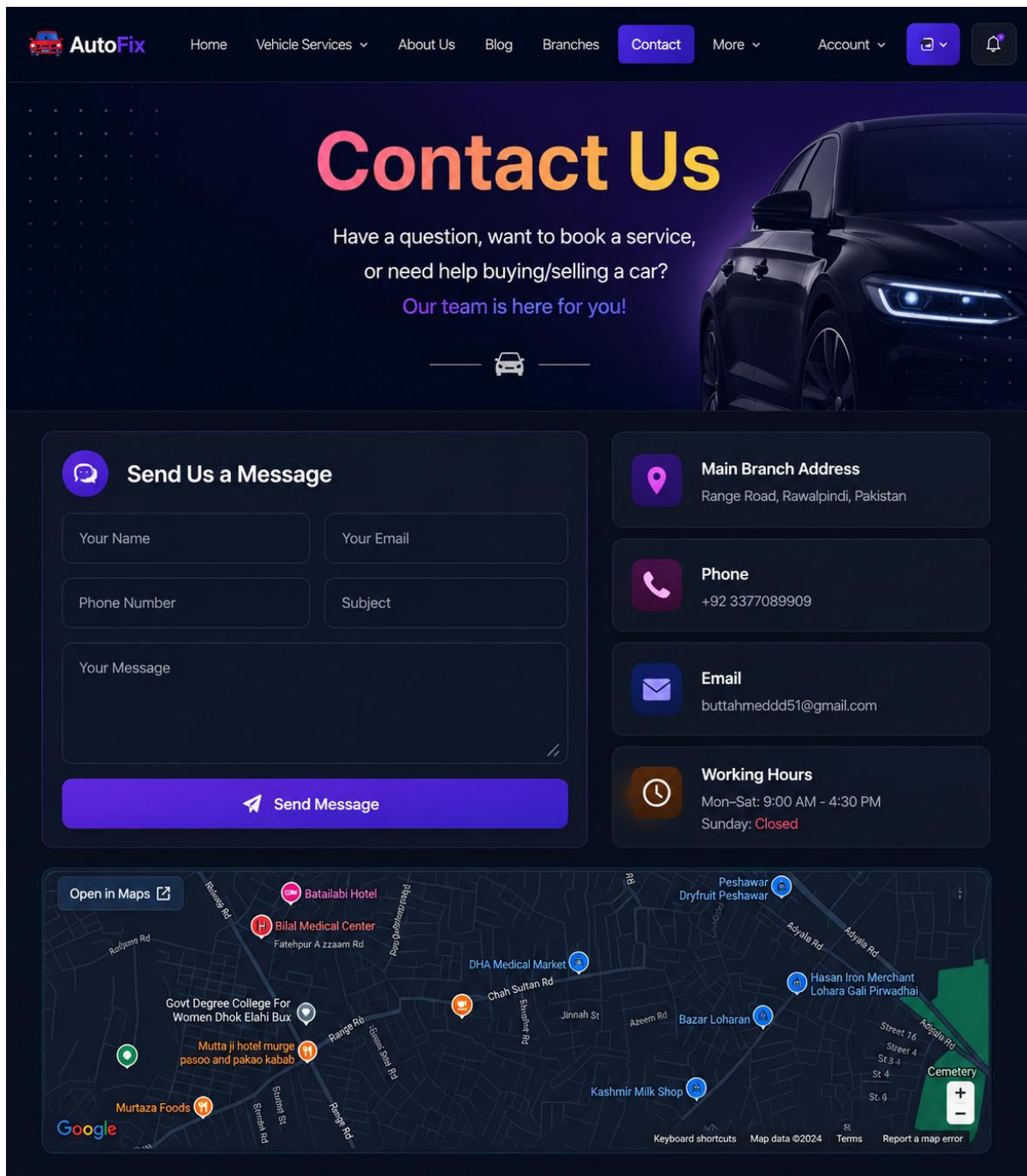


Figure 6.5: Contact Us Page Screen

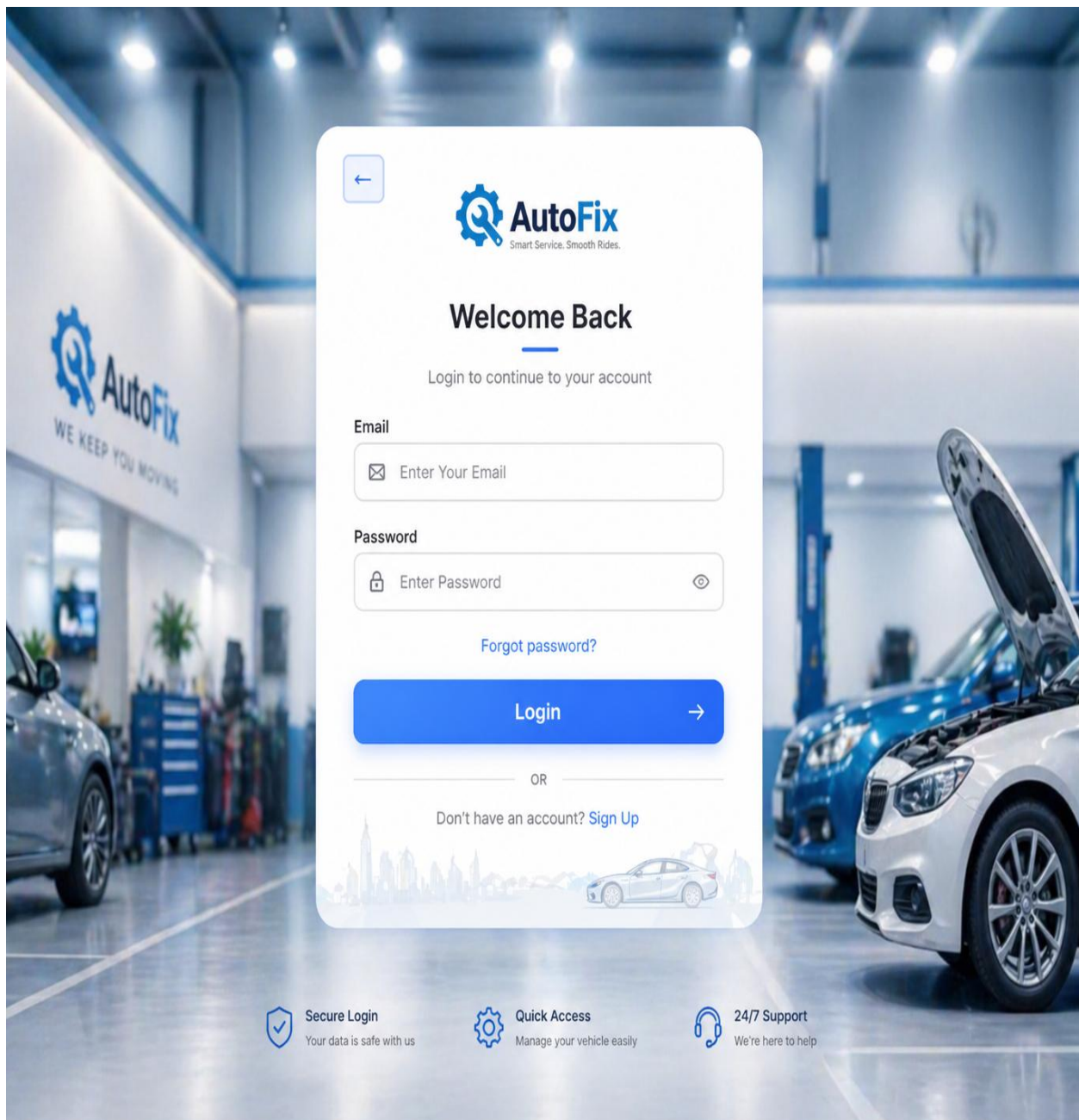


Figure 6.6: Login Screen

 **Vehicle Maintenance Prediction** 

VEHICLE INFORMATION

Vehicle Model

Honda Civic

Mileage (km)

50000

Vehicle Age (years)

5

MAINTENANCE DETAILS

Maintenance History (0–5)

0 = Poor, 5 = Excellent

Fuel Efficiency (km/L)

15

VEHICLE CONDITION

Odometer (km)

50000

Accident History

No accidents

COMPONENT STATUS (0=POOR, 1=FAIR, 2=GOOD)

Brake Condition

Good

Battery Status

Good

Tire Condition

Good

 **Analyze Vehicle**

Reset Form

Figure 6.7: Vehicle Maintenance Prediction Tool Screen

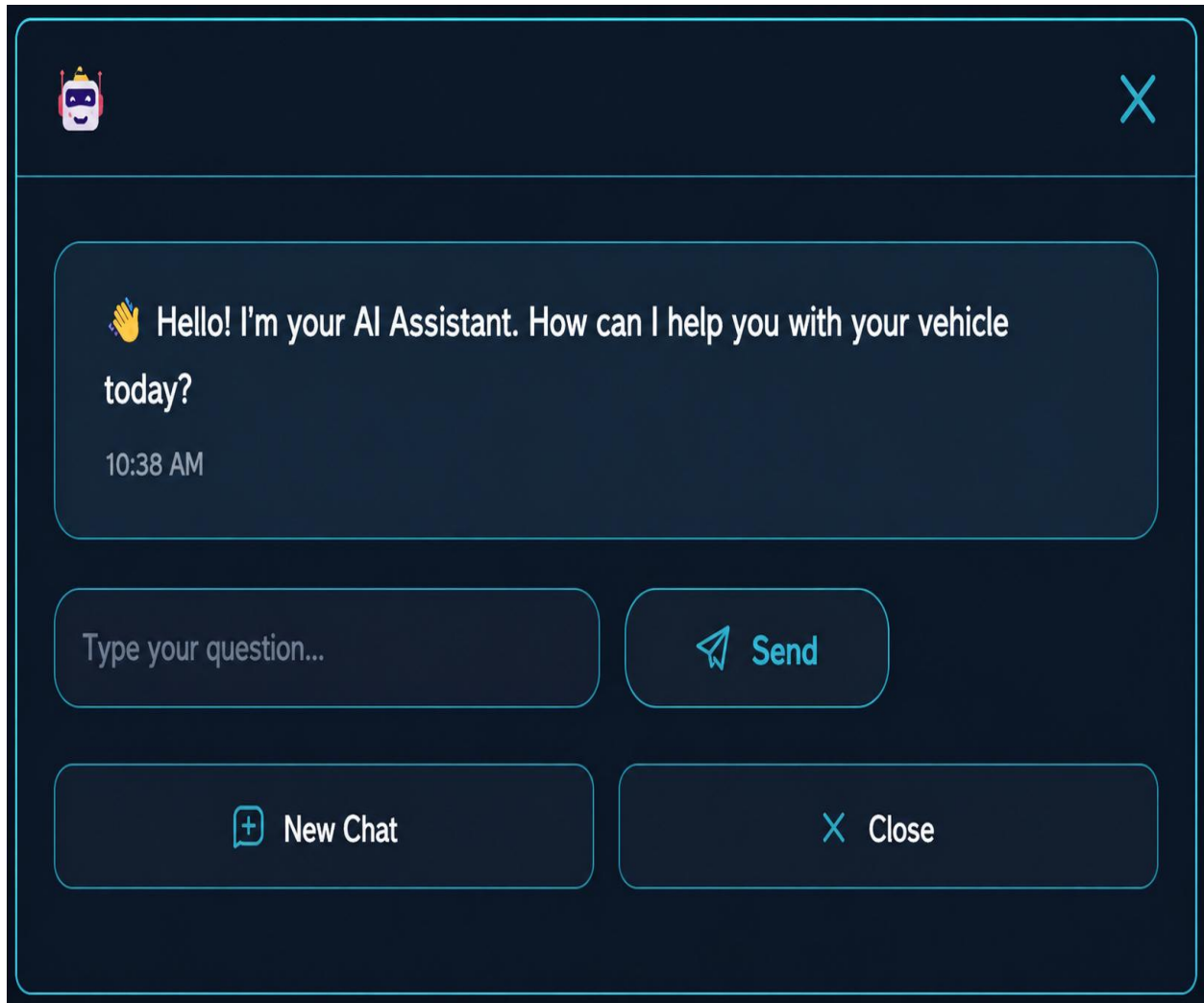


Figure 6.8: AI Assistant Chatbot Screen

Payment

X

Complete Your Payment

Amount:

PKR 2,200

Secure, fast checkout with Link ▼

Card number

Expiration date

Security code

1234 1234 1234 1234

MC

VISA

AMEX

MM / YY

CVC

Country

Pakistan ▼

☐ Save this card for future payments

Pay PKR 2,200

Figure 6.9: Payment Modal Screen

- **Role-Based Dashboards (Post-Login)**

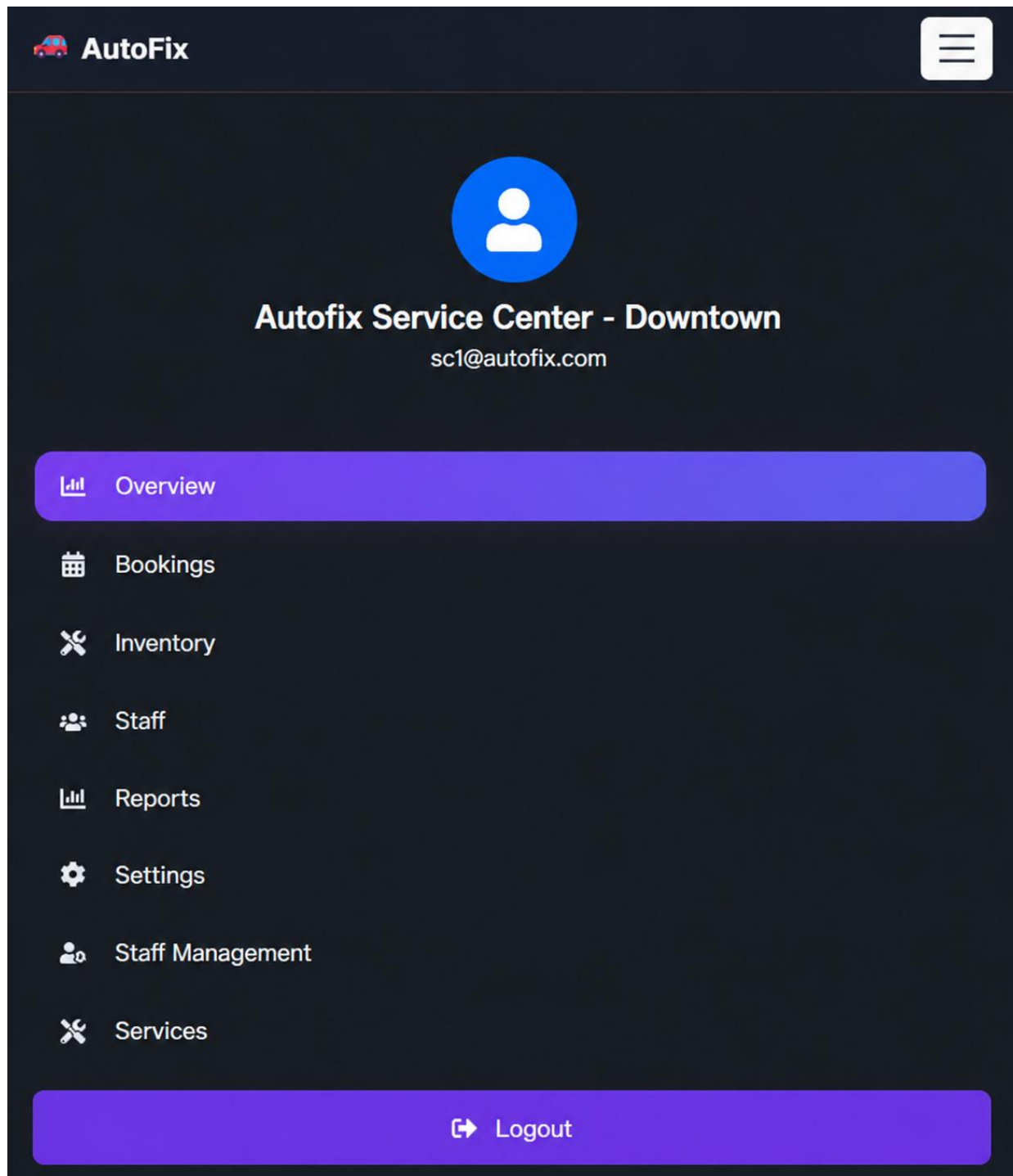


Figure 6.10: Admin Dashboard Sidebar

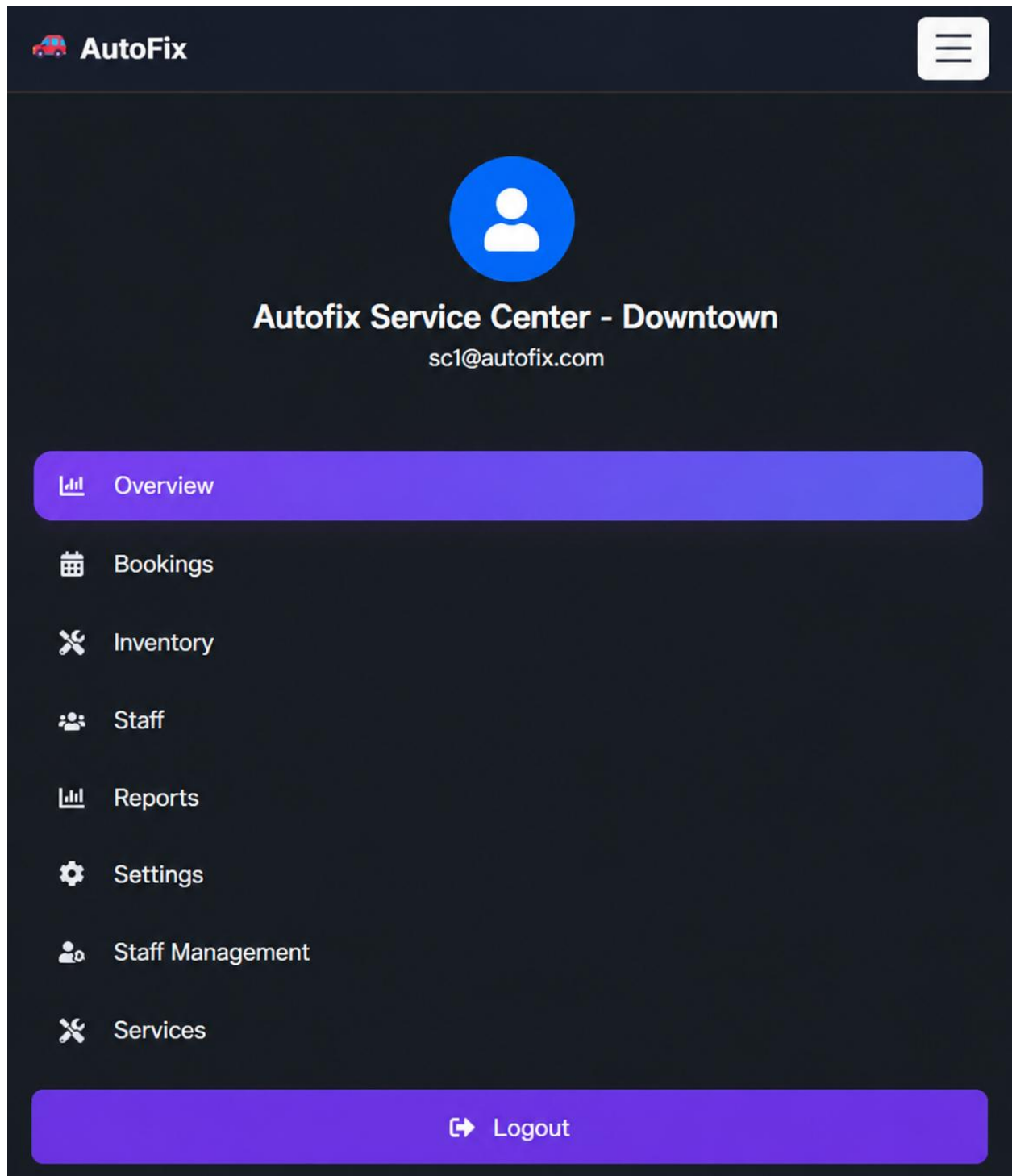


Figure 6.11: Manager Dashboard Sidebar

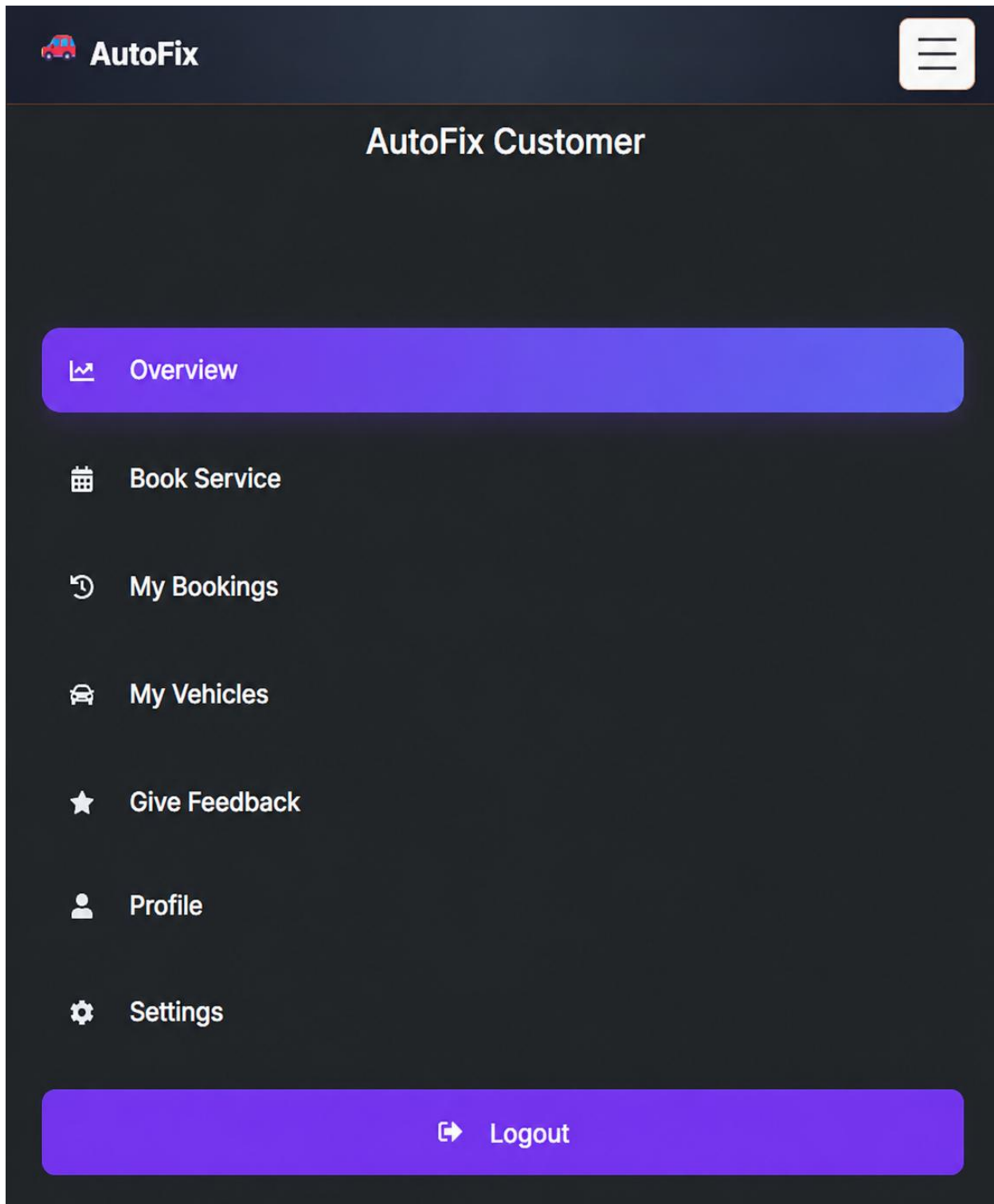


Figure 6.12: Customer Dashboard Sidebar

7 REFERENCES

- [1] React.js Documentation, Meta Platforms Inc. Available: <https://react.dev>
- [2] Django REST Framework Documentation. Available: <https://www.django-rest-framework.org>
- [3] MySQL Documentation, Oracle Corporation. Available: <https://www.mysql.com>
- [4] Scikit-learn Machine Learning Library Documentation. Available: <https://scikit-learn.org>
- [5] Chart.js Documentation. Available: <https://www.chartjs.org>
- [6] Redis Documentation. Available: <https://redis.io/docs>
- [7] JSON Web Tokens (JWT) Introduction. Available: <https://jwt.io/introduction>
- [8] Bootstrap Documentation. Available: <https://getbootstrap.com>
- [9] Tailwind CSS Documentation. Available: <https://tailwindcss.com>
- [10] Python Documentation. Available: <https://www.python.org/doc>